Formal Verification of Software/Hardware Systems

The Need of Verification

Technologies

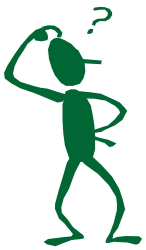
AI

Learning

Big Data

Cyber Physical Systems (Modern Embedded Systems)

Parallel Systems: ASIC, FPGA, GPU

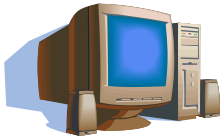


Embedded Systems: Everywhere!

- Computational
- Integral with physical processes
  - sensors
- Reactive
  - at the speed of the environment
- Heterogeneous
  - hardware/software, mixed architectures
- Networked
  - shared, adaptive

Embedded Software

- 80% of all software is embedded
- Demands for increased functionality with minimal resources
- Requires multitude of skills
  - Software construction
  - Hardware platforms,
  - Communication
  - Automation
  - Testing & Verification
- Goal:
  - Give a qualitative lift to current industrial practice









Design Challenges: Complexity

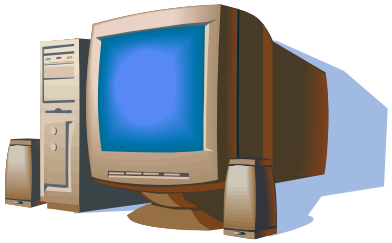
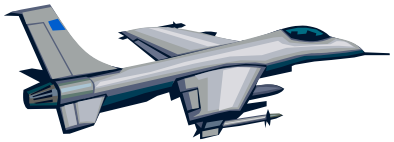
- From the recent NASA's data, the size of software code is doubling every 3-4 years.
- With the rapidly increasing demand for the large scale and complex embedded software,
  - The likely number of design errors *grow exponentially* with the number of interacting system components.
- A major challenge of software engineering:
  - enable developers to construct systems that operate reliably despite this complexity.

Software is Full of Bugs !

- "Software easily rates as the most poorly constructed, unreliable, and least maintainable technological artifacts invented by man" *Paul Strassman, former CIO of Xerox*

- In the NASA critical mission software error statistics,
  - 0.025 errors with 30K,
  - 0.4 errors with 100K,
  - 6.3 errors with 1Mb code
  - 100 mission critical errors with 10Mb.
  - Clearly, it presents an unacceptable trend.

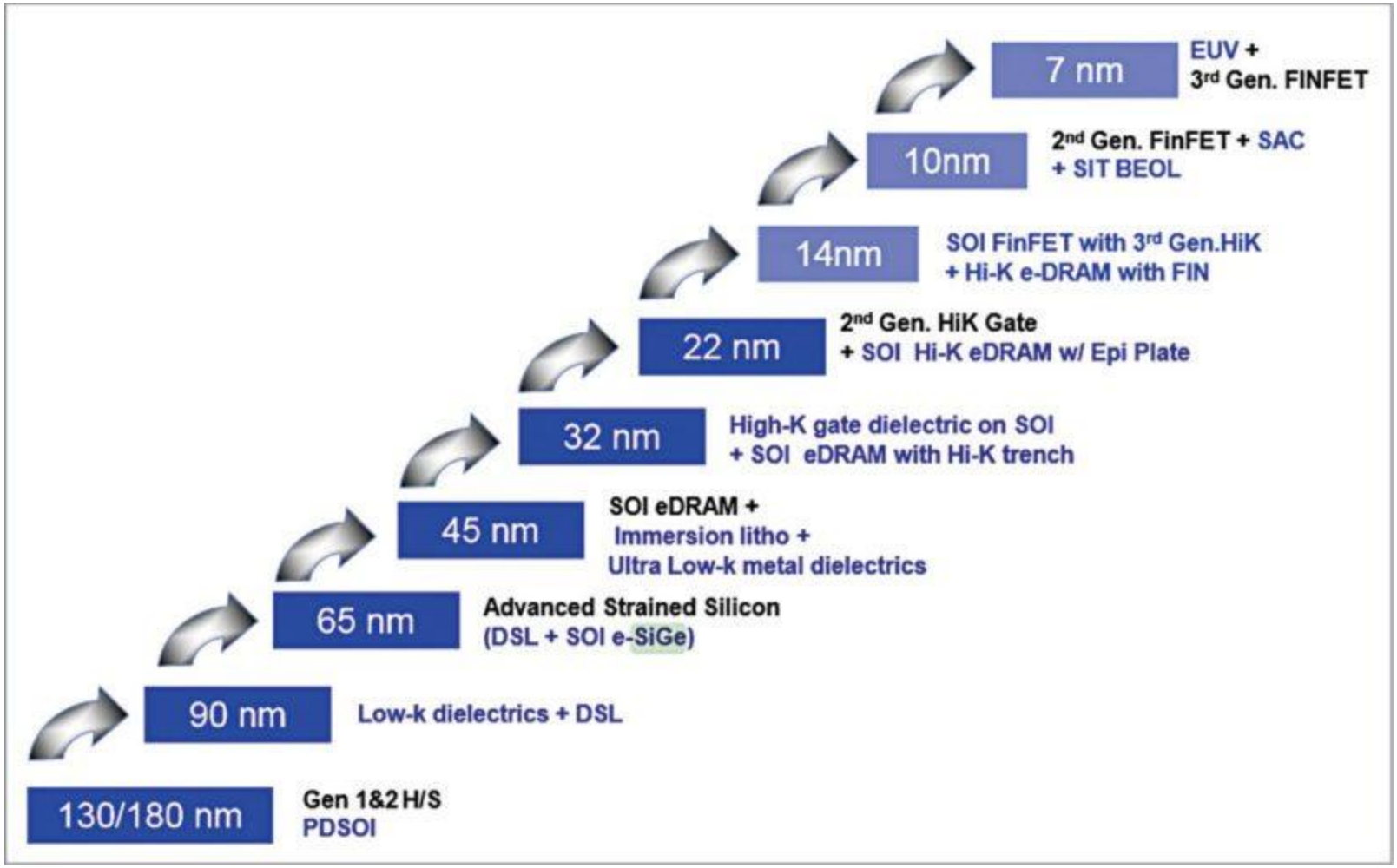
Hardware is Everywhere



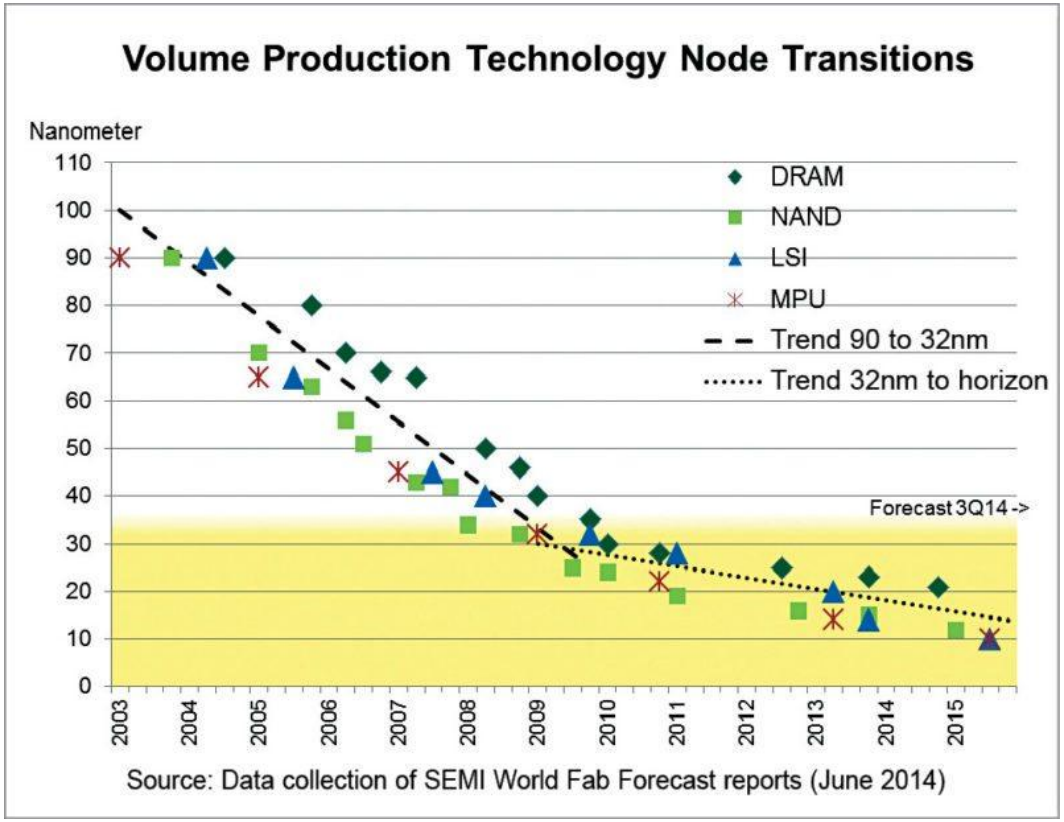



Typical Device Sizes



Typical Device Sizes ≈ 10nm (a plateau)

- Digital devices: 5nm
- Analog devices: 5nm
- RF devices: 28nm



Functional Validation of Microprocessors

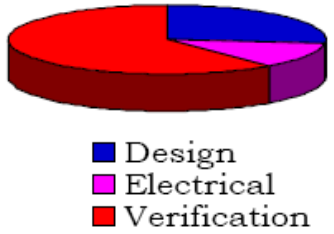
- Functional validation is a major bottleneck
  - Deeply pipelined complex micro-architectures
- Logic bugs increase at 3-4 times/generation
  - Bugs increase (exponential) is linear with design complexity growth

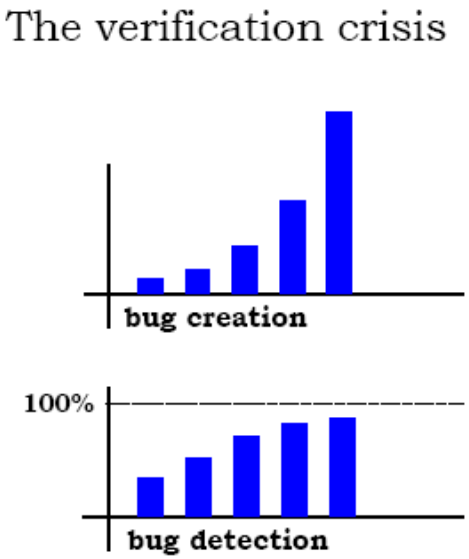
Bugs

- What is a "bug"?
  - When the design does not match the specification
- Problem is that complete (and consistent) specifications may not exist for many products

Verification Challenges

- Functional Correctness Problem:
  - 71% of Re-spins (Source: 2002 Collett International)
  - The first cause of Re-spins
- For ASIC Designs
  - 70% of time devoted to the design verification
  - 30% of time devoted to the design
- Leading microprocessor design groups:
  - One half of the design resources of a project, including computers and manpower, to be devoted to verification.





Bugs are Expensive!

- Examples:
  - Y2K bug: Estimated cost: >$500 Billion
  - Northeast Blackout of 2003:
    - A programming error identified as the cause of alarm failure
    - Estimated cost: $6-$10 Billion
  - The notorious Pentium FDIV bug over two decades ago:
    - Estimated cost: $500 Million.
- "The cost of software bugs to the U.S. economy is estimated at $60 B/year" NIST, 2002

Bugs are Catastrophic!

- On June 4, 1996 an Ariane V rocket launched by the rocket launched by the European Space Agency European Space Agency exploded just forty seconds exploded just forty seconds after lift-off
- Damage: half a billion dollars

Bugs are Catastrophic!

- Therac-25 was a radiation therapy machine produced by Atomic Energy of Canada Limited (AECL) and CGR of France after the Therac-6 and Therac-20 units.
- It was involved with at least six known accidents between 1985 and 1987, in which patients were given massive overdoses of radiation.
- At least five patients died of the overdoses.
- These accidents highlighted the dangers of software control of safety-critical systems, and they have become a standard case study in health informatics.

Meltdown & Spectre Vulnerabilities (2018)

- The way that most modern CPUs work, including silicon from Intel, AMD and ARM, is that in order to improve performance, they make use of a technique known as speculative execution.
- With speculative execution, the CPU predicts what will be needed next for a given process.
- With the Meltdown and Spectre flaws, an attacker could theoretically abuse the speculative execution model to read system memory and potentially get unauthorized access to information.

- Meltdown and Spectre: 'worst ever' CPU bugs affect virtually all computers
- Intel botched its first major bug.

Design Process

Design : specify and enter the design intent

Verify:

verify the correctness of design and implementation

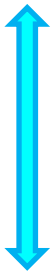
Implement:

refine the design through all phases

What is Verification?

Specification and Verification

Specification



Verification

Implementation

Verification

- Traditional Verification (Dynamic Verification)
  - Simulation, Testing.
- Formal verification
  - Static verification, symbolic verification., etc.

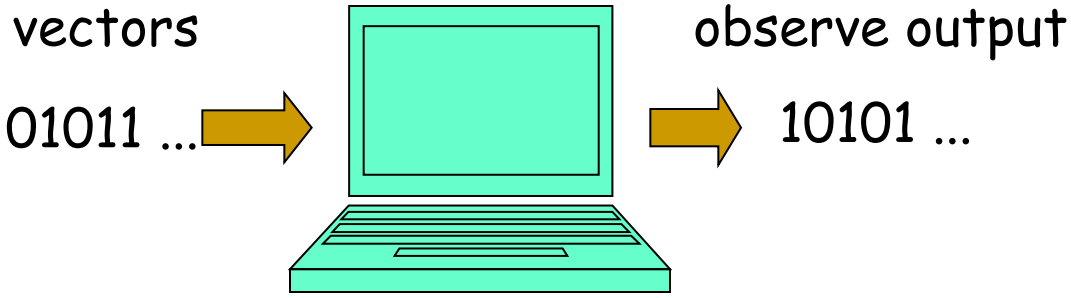
Simulation

- Most widely used analysis technique.
- From a technical point of view just a "walk" in the reachability graph.
- By making many "walks" or a very "long walk" behavior, it is possible to make reliable statements about properties/ performance indicators.
- Used for validation and performance analysis.
- Cannot be used to prove correctness!

- $(x+1)^2 = x^2 + 2x + 1$?

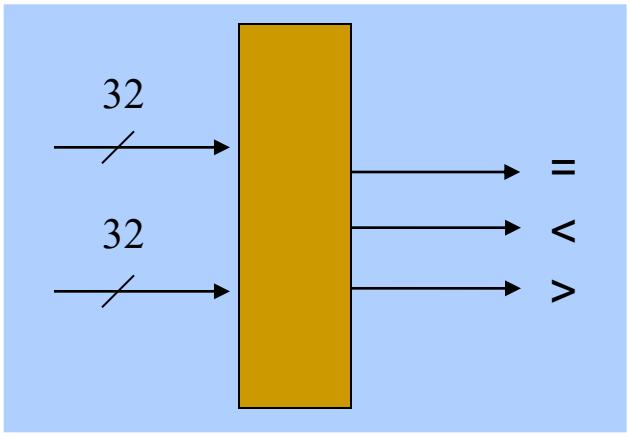
x	$(x+1)^2$	x^2+2x+1
0	1	1
1	4	4
2	9	9
3	16	16
9	100	100
67	4624	4624
...	...	...

How About Testing/Simulation?



- Long test runs. Need to simulate billions of cycles
- Effective test sequences hard to generate
- Quality of input patterns limits reliability
- Incomplete: no guarantee for sequences not simulated
- High-coverage, Corner cases
- Errors often occur where not anticipated

Exhaustive Testing/Simulation?

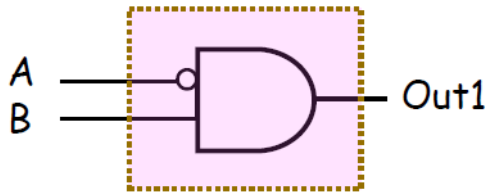


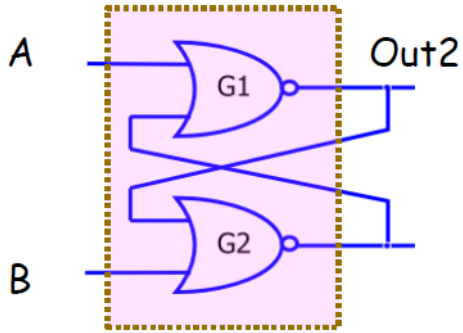
A fast computer: 1 micro second per vector

$$\frac{2^{64}\ \text{(all input patterns)} \times 10^{-6}}{3600\ \text{(seconds)} \times 24\ \text{(hours)} \times 365\ \text{(days)}}$$

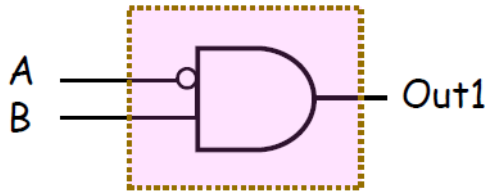
Exhaustive Simulation: Not Sufficient

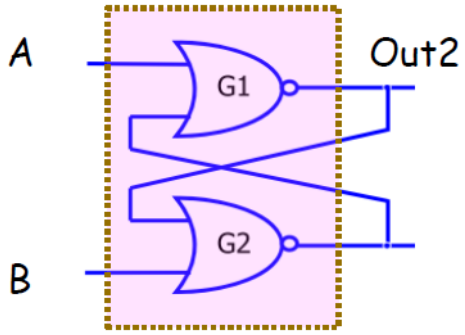
- Verify C1 (Specification) against C2 (Implementation).





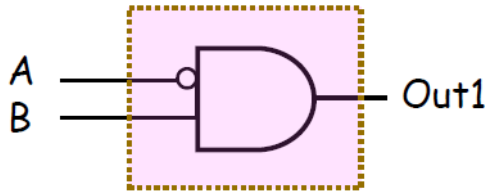
Exhaustive Simulation: Not Sufficient

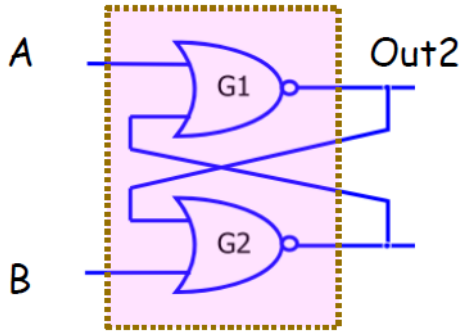




- Verify C1 (Specification) against C2 (Implementation).
- Exhaustive Simulation 1: Correct !
  - A B Out1 Out2
  - 1 0 0 0
  - 0 0 0 0
  - 1 1 0 0
  - 0 1 1 1

Exhaustive Simulation: Not Sufficient





- Verify C1 (Specification) against C2 (Implementation).
- Exhaustive Simulation 2: Incorrect !
  - A B Out1 Out2
  - 1 0 0 0
  - 1 1 0 0
  - 0 1 1 1
  - 0 0 0 1

A Circuit Netlist: Combinational c432.bench of 250 Gates

```
INPUT(G1)
INPUT(G2)
INPUT(G3)
INPUT(G4)
INPUT(G5)
INPUT(G6)
INPUT(G7)
INPUT(G8)
INPUT(G9)
INPUT(G10)
INPUT(G11)
INPUT(G12)
INPUT(G13)
INPUT(G14)
INPUT(G15)
INPUT(G16)
INPUT(G17)
INPUT(G18)
INPUT(G19)
INPUT(G20)
INPUT(G21)
INPUT(G22)
INPUT(G23)
INPUT(G24)
INPUT(G25)
INPUT(G26)
INPUT(G27)
INPUT(G28)
INPUT(G29)
INPUT(G30)
INPUT(G31)
INPUT(G32)
INPUT(G33)
INPUT(G34)
INPUT(G35)
INPUT(G36)
OUTPUT(G426)
OUTPUT(G427)
OUTPUT(G428)
OUTPUT(G429)
OUTPUT(G430)
OUTPUT(G431)
OUTPUT(G432)
G118 = NOT(G1)
G119 = NOT(G2)
G122 = NOT(G4)
G123 = NOT(G6)
G126 = NOT(G8)
G127 = NOT(G10)
G130 = NOT(G12)
G131 = NOT(G14)
G134 = NOT(G16)
G135 = NOT(G18)
G138 = NOT(G20)
G139 = NOT(G22)
G142 = NOT(G24)
G143 = NOT(G26)
G146 = NOT(G28)
G147 = NOT(G30)
G150 = NOT(G32)
G151 = NOT(G34)
G154 = NAND(G118,G2)
G157 = NOR(G3,G119)
G158 = NOR(G5,G119)
G159 = NAND(G122,G6)
G162 = NAND(G126,G10)
G165 = NAND(G130,G14)
G168 = NAND(G134,G18)
G171 = NAND(G138,G22)
G174 = NAND(G142,G26)
G177 = NAND(G146,G30)
G180 = NAND(G150,G34)
G183 = NOR(G7,G123)
G184 = NOR(G9,G123)
G185 = NOR(G11,G127)
G186 = NOR(G13,G127)
G187 = NOR(G15,G131)
G188 = NOR(G17,G131)
G189 = NOR(G19,G135)
G190 = NOR(G21,G135)
G191 = NOR(G23,G139)
G192 = NOR(G25,G139)
G193 = NOR(G27,G143)
G194 = NOR(G29,G143)
G195 = NOR(G31,G147)
G196 = NOR(G33,G147)
G197 = NOR(G35,G151)
G198 = NOR(G36,G151)
G199 = AND(G154,G159,G162,G165,G168,G171,G174,G177,G180)
G203 = NOT(G199)
G213 = NOT(G199)
# G223 = XOR(G203,G154)
W223 = NOT(G203)
Z223 = NOT(G154)
X223 = NAND(G203,Z223)
Y223 = NAND(G154,W223)
G223 = NAND(X223,Y223)
# G226 = XOR(G203,G159)
W226 = NOT(G203)
Z226 = NOT(G159)
X226 = NAND(G203,Z226)
Y226 = NAND(G159,W226)
G226 = NAND(X226,Y226)
# G229 = XOR(G203,G162)
W229 = NOT(G203)
Z229 = NOT(G162)
X229 = NAND(G203,Z229)
Y229 = NAND(G162,W229)
G229 = NAND(X229,Y229)
# G232 = XOR(G203,G165)
W232 = NOT(G203)
Z232 = NOT(G165)
X232 = NAND(G203,Z232)
Y232 = NAND(G165,W232)
G232 = NAND(X232,Y232)
# G235 = XOR(G203,G168)
W235 = NOT(G203)
Z235 = NOT(G168)
X235 = NAND(G203,Z235)
Y235 = NAND(G168,W235)
G235 = NAND(X235,Y235)
# G238 = XOR(G203,G171)
W238 = NOT(G203)
Z238 = NOT(G171)
X238 = NAND(G203,Z238)
Y238 = NAND(G171,W238)
G238 = NAND(X238,Y238)
G241 = NAND(G1,G213)
# G242 = XOR(G203,G174)
W242 = NOT(G203)
Z242 = NOT(G174)
X242 = NAND(G203,Z242)
Y242 = NAND(G174,W242)
G242 = NAND(X242,Y242)
G245 = NAND(G213,G4)
# G246 = XOR(G203,G177)
W246 = NOT(G203)
Z246 = NOT(G177)
X246 = NAND(G203,Z246)
Y246 = NAND(G177,W246)
G246 = NAND(X246,Y246)
G249 = NAND(G213,G8)
# G250 = XOR(G203,G180)
W250 = NOT(G203)
Z250 = NOT(G180)
X250 = NAND(G203,Z250)
Y250 = NAND(G180,W250)
G250 = NAND(X250,Y250)
G253 = NAND(G213,G12)
G254 = NAND(G213,G16)
G255 = NAND(G213,G20)
G256 = NAND(G213,G24)
G257 = NAND(G213,G28)
G258 = NAND(G213,G32)
G259 = NAND(G223,G157)
G262 = NAND(G223,G158)
G263 = NAND(G226,G183)
G266 = NAND(G229,G185)
G269 = NAND(G232,G187)
G272 = NAND(G235,G189)
G275 = NAND(G238,G191)
G278 = NAND(G242,G193)
G281 = NAND(G246,G195)
G284 = NAND(G250,G197)
G287 = NAND(G226,G184)
G288 = NAND(G229,G186)
G289 = NAND(G232,G188)
G290 = NAND(G235,G190)
G291 = NAND(G238,G192)
G292 = NAND(G242,G194)
G293 = NAND(G246,G196)
G294 = NAND(G250,G198)
G295 = AND(G259,G263,G266,G269,G272,G275,G278,G281,G284)
G299 = NOT(G262)
G300 = NOT(G287)
G301 = NOT(G288)
G302 = NOT(G289)
G303 = NOT(G290)
G304 = NOT(G291)
G305 = NOT(G292)
G306 = NOT(G293)
G307 = NOT(G294)
G308 = NOT(G295)
G318 = NOT(G295)
# G328 = XOR(G308,G259)
W328 = NOT(G308)
Z328 = NOT(G259)
X328 = NAND(G308,Z328)
Y328 = NAND(G259,W328)
G328 = NAND(X328,Y328)
# G329 = XOR(G308,G263)
W329 = NOT(G308)
Z329 = NOT(G263)
X329 = NAND(G308,Z329)
Y329 = NAND(G263,W329)
G329 = NAND(X329,Y329)
# G330 = XOR(G308,G266)
W330 = NOT(G308)
Z330 = NOT(G266)
X330 = NAND(G308,Z330)
Y330 = NAND(G266,W330)
G330 = NAND(X330,Y330)
# G331 = XOR(G308,G269)
W331 = NOT(G308)
Z331 = NOT(G269)
X331 = NAND(G308,Z331)
Y331 = NAND(G269,W331)
G331 = NAND(X331,Y331)
G332 = NAND(G3,G318)
# G333 = XOR(G308,G272)
W333 = NOT(G308)
Z333 = NOT(G272)
X333 = NAND(G308,Z333)
Y333 = NAND(G272,W333)
G333 = NAND(X333,Y333)
G334 = NAND(G318,G7)
# G335 = XOR(G308,G275)
W335 = NOT(G308)
Z335 = NOT(G275)
X335 = NAND(G308,Z335)
Y335 = NAND(G275,W335)
G335 = NAND(X335,Y335)
G336 = NAND(G318,G11)
# G337 = XOR(G308,G278)
W337 = NOT(G308)
Z337 = NOT(G278)
X337 = NAND(G308,Z337)
Y337 = NAND(G278,W337)
G337 = NAND(X337,Y337)
G338 = NAND(G318,G15)
# G339 = XOR(G308,G281)
W339 = NOT(G308)
Z339 = NOT(G281)
X339 = NAND(G308,Z339)
Y339 = NAND(G281,W339)
G339 = NAND(X339,Y339)
G340 = NAND(G318,G19)
# G341 = XOR(G308,G284)
W341 = NOT(G308)
Z341 = NOT(G284)
X341 = NAND(G308,Z341)
Y341 = NAND(G284,W341)
G341 = NAND(X341,Y341)
G342 = NAND(G318,G23)
G343 = NAND(G318,G27)
G344 = NAND(G318,G31)
G345 = NAND(G318,G35)
G346 = NAND(G328,G299)
G347 = NAND(G329,G300)
G348 = NAND(G330,G301)
G349 = NAND(G331,G302)
G350 = NAND(G333,G303)
G351 = NAND(G335,G304)
G352 = NAND(G337,G305)
G353 = NAND(G339,G306)
G354 = NAND(G341,G307)
G355 = AND(G346,G347,G348,G349,G350,G351,G352,G353,G354)
G358 = NOT(G355)
G368 = NAND(G5,G358)
G369 = NAND(G358,G9)
G370 = NAND(G358,G13)
G371 = NAND(G358,G17)
G372 = NAND(G358,G21)
G373 = NAND(G358,G25)
G374 = NAND(G358,G29)
G375 = NAND(G358,G33)
G376 = NAND(G358,G36)
G377 = NAND(G2,G241,G332,G368)
G378 = NAND(G245,G334,G369,G6)
G383 = NAND(G249,G336,G370,G10)
G390 = NAND(G253,G338,G371,G14)
G396 = NAND(G254,G340,G372,G18)
G401 = NAND(G255,G342,G373,G22)
G404 = NAND(G256,G343,G374,G26)
G408 = NAND(G257,G344,G375,G30)
G411 = NAND(G258,G345,G376,G34)
G412 = NOT(G377)
G413 = AND(G378,G383,G390,G396,G401,G404,G408,G411)
G414 = NOT(G390)
G415 = NOT(G401)
G416 = NOT(G404)
G417 = NOT(G408)
G418 = NAND(G383,G414)
G421 = NAND(G383,G390,G415,G396)
G424 = NAND(G396,G390,G416)
G425 = NAND(G383,G390,G404,G417)
G426 = NOT(G199)
G427 = NOT(G295)
G428 = NOT(G355)
G429 = NOR(G412,G413)
G430 = NAND(G378,G383,G418,G396)
G431 = NAND(G378,G383,G421,G424)
G432 = NAND(G378,G418,G421,G425)
```

When Do You Tapeout?

- Motorola criteria
  - 40 billion random cycles without finding a bug
  - Directed tests in verification plan are completed
  - Source code and/or functional coverage goals are met
  - Diminishing bug rate is observed
  - A certain date on the calendar is reached

What is "Formal" Verification?

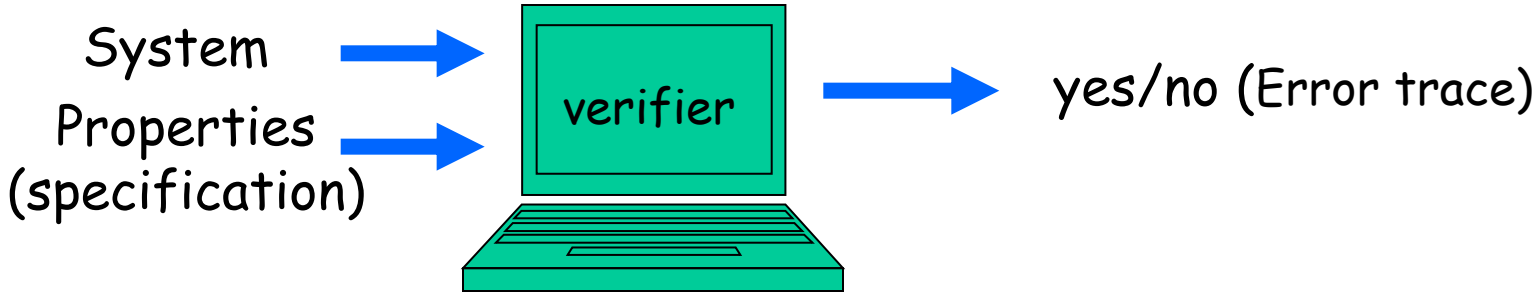
Formal Verification (Formal Methods)

- Mathematically-based techniques
- Provides a framework for
  - Specifying systems (and thus notion of correctness)
  - Ensuring correctness
    - implementation w.r.t. the specification
    - equivalence of different implementations
- Reasoning is based on logic
  - Amenable to machine analysis and manipulation
  - Can verify everything that is true in the system!

Formal Verification

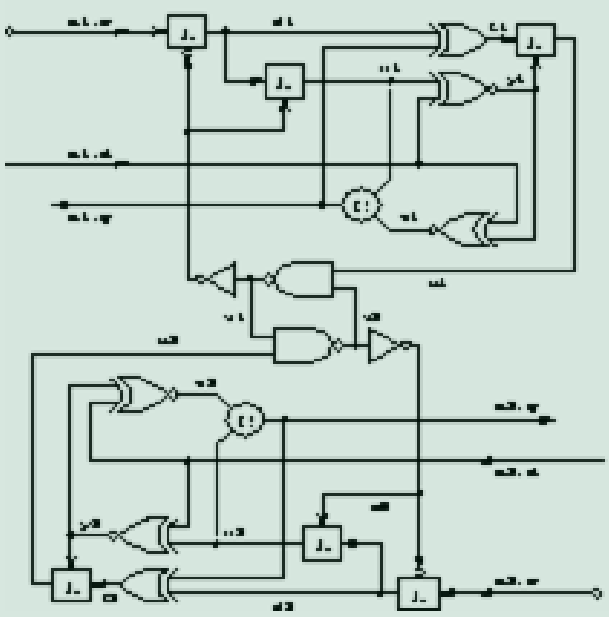
- Techniques and tools for specifying and verifying such systems.
- Greatly increase our understanding of a system by revealing
  - inconsistencies,
  - ambiguities and
  - incompletenesses

Formal Verification Methods



- Complete with respect to a given property
- Correctness is guaranteed mathematically
- Can generate a short trace if a property fails
- Formal verification is useful to find bugs in design
- Consideration of all cases is implicit in formal verification.

The Most Cited Example



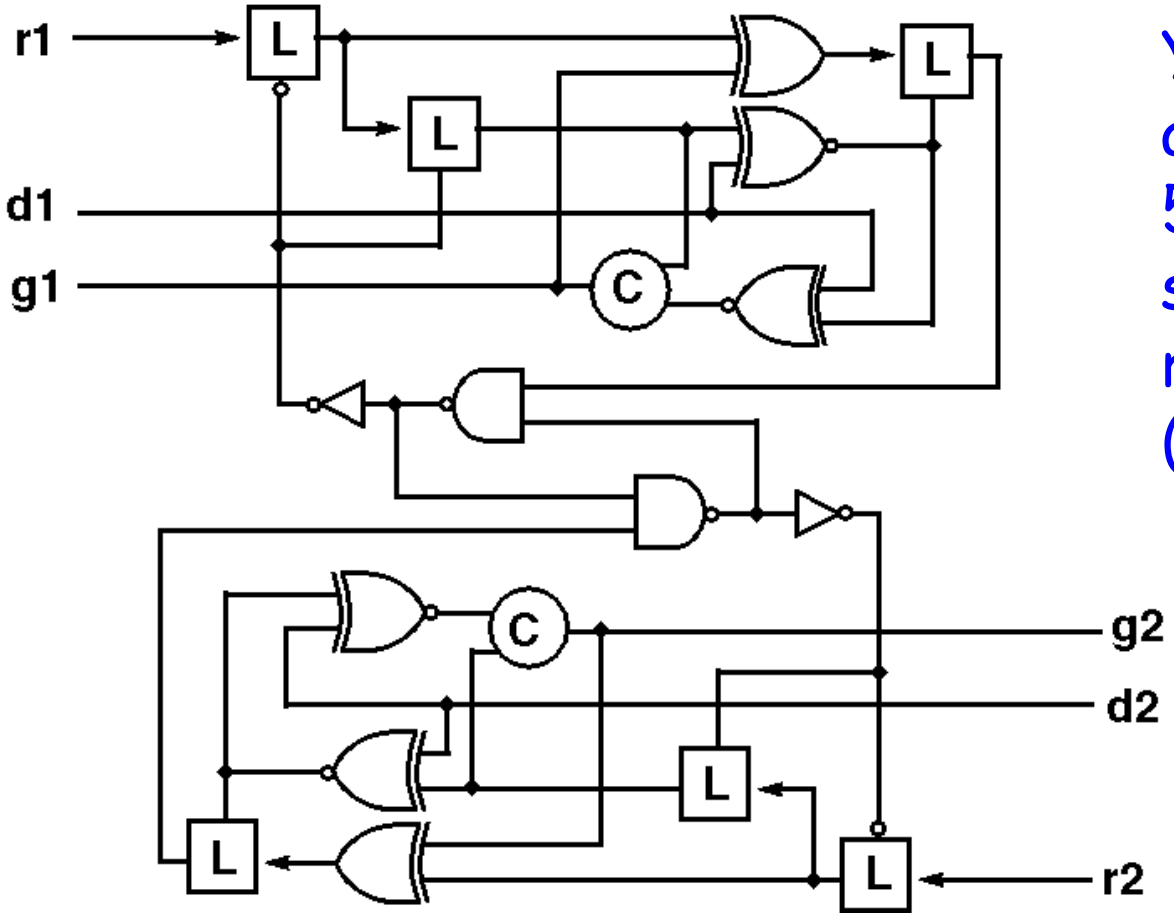
100 hours of simulation:

NO ERRORS FOUND

5 min of design verification

AN ERROR IS FOUND

A Correct Transition Arbiter?



YES!
according to
50 hours of
simulation using
random delays.
(2^{31} transitions)

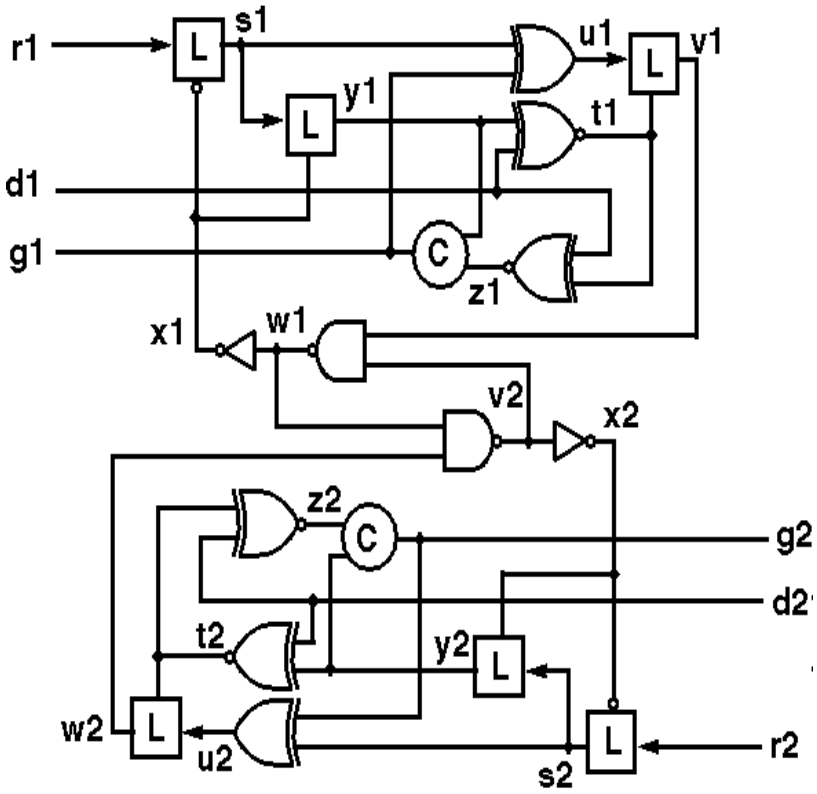
A Correct Transition Arbiter?

Symbolic model checking says NO!!!! in < 1 minute!
Error trace:

```
r1,d1,g1, ... ,g2,d2,r2
000000010011001000000 0->
r1->s1->u1->v1->r2->s2->u2->
w2->v2->x2->y2->t2->z2->g2->
d2->r2->u2->z2->t2->w2->v2->
x2->s2->u2->w2->v2->x2->y2->
g2->u2->w2->v2->w1->x1->y1->
t1->z1->g1->
101111101101001000 1010
```

Client 1 granted resource

Client 2 granted resource



Formal Verification: Software->Hardware->Software

- In the 1960-70's, high expectations for "software verification", but hopes gradually fizzled out by the late 1970's.
  - Theorem proving approaches have "cultural roots" in software verification in 1970's.

- Why formal methods might work for hardware?
  - Hardware is often regular and hierarchical
  - Re-use of design is common practice
  - Primitives are simpler

Formal Verification

- Two well established approaches to verification are
  - model checking
  - theorem proving.

- 25 years of model checking
  - Increasing acceptance by HW industry
  - Significant recent progress in applications to SW.
  - Main challenge: state-explosion problem

State of The Art

- In the past the use of formal methods in practice seemed hopeless.
- The notations were too obscure the techniques did not scale and the tool support was inadequate or too hard to use.
- There were only a few nontrivial case studies and together they still were not convincing enough to the practicing software or hardware engineer

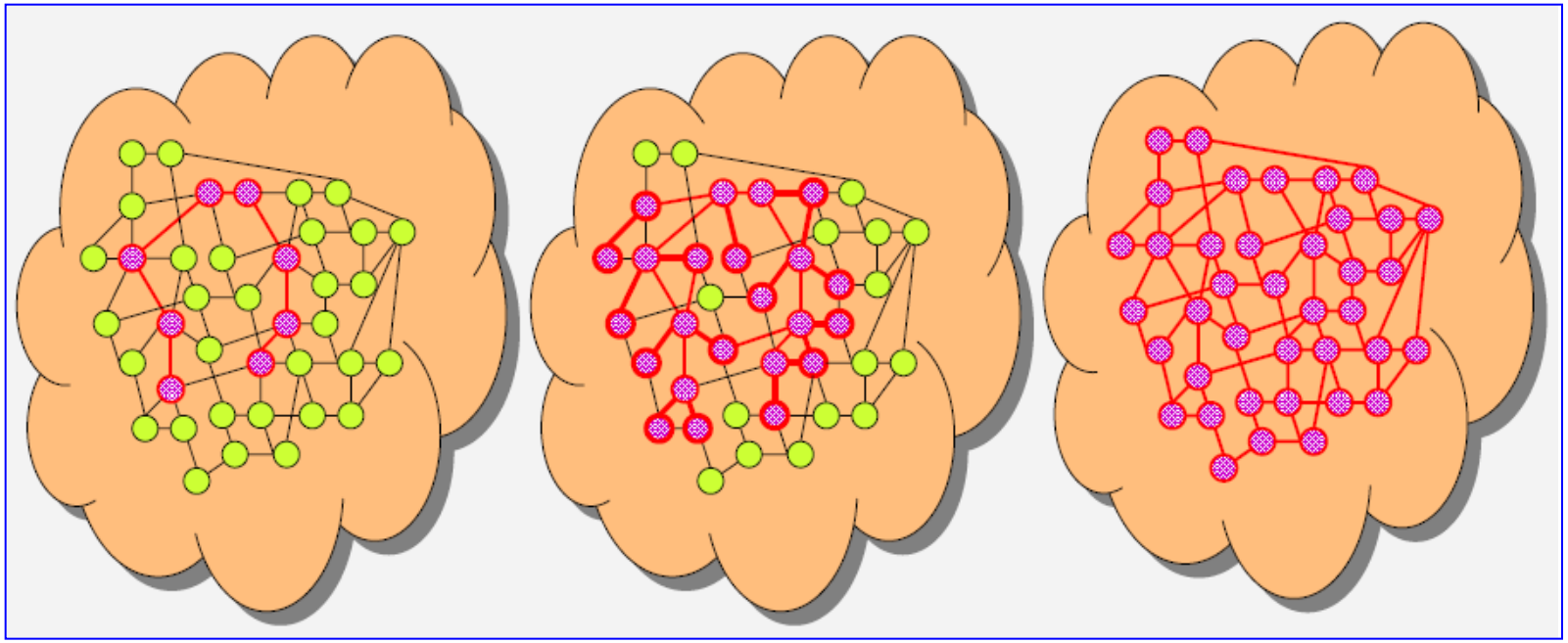
- Digital system verification
- Protocol verification
- Embedded software verification
- Security validation
- Program verification
- ……

Formal Verification

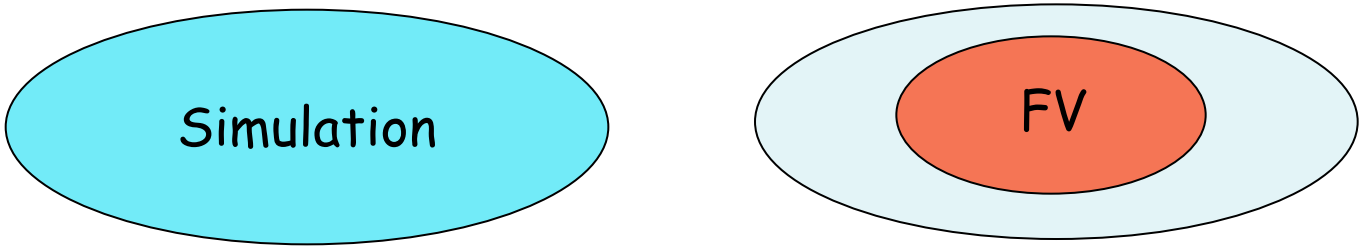
- Wide impact on any complex systems
- Across the domains.
- Long lasting Knowledge

Difference Between Formal Verification and Simulation

Simulation, Semi-Formal, Formal Verification



Simulation/Test vs. FV

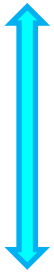


- Simulation: complete model, partial verification
- Verification: partial model, complete verification
- Formal verification cannot replace simulation
  - Current technology only effective for certain verification subtasks
- Two techniques are complementary
- Formal verification gives additional confidence

Informal to Formal Specification

Specification and Verification

Specification



Verification

Implementation

Specification

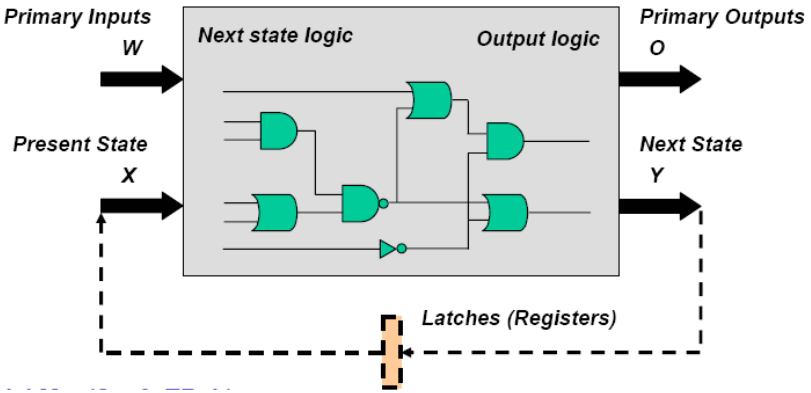
- The process of specification is the act of writing things down precisely.
- The main benefit in so doing is intangible:
  - gaining a deeper understanding of the system being specified.
- It is through this specification process that developers uncover design
  - flaws
  - inconsistencies
  - ambiguities and
  - incompletenesses

Formal Specification

- Specification is the process of describing a system and its desired properties.
- Formal specification uses a language with a mathematically defined syntax and semantics.
- The system properties might include
  - functional behavior
  - timing behavior
  - performance characteristics or
  - internal structure

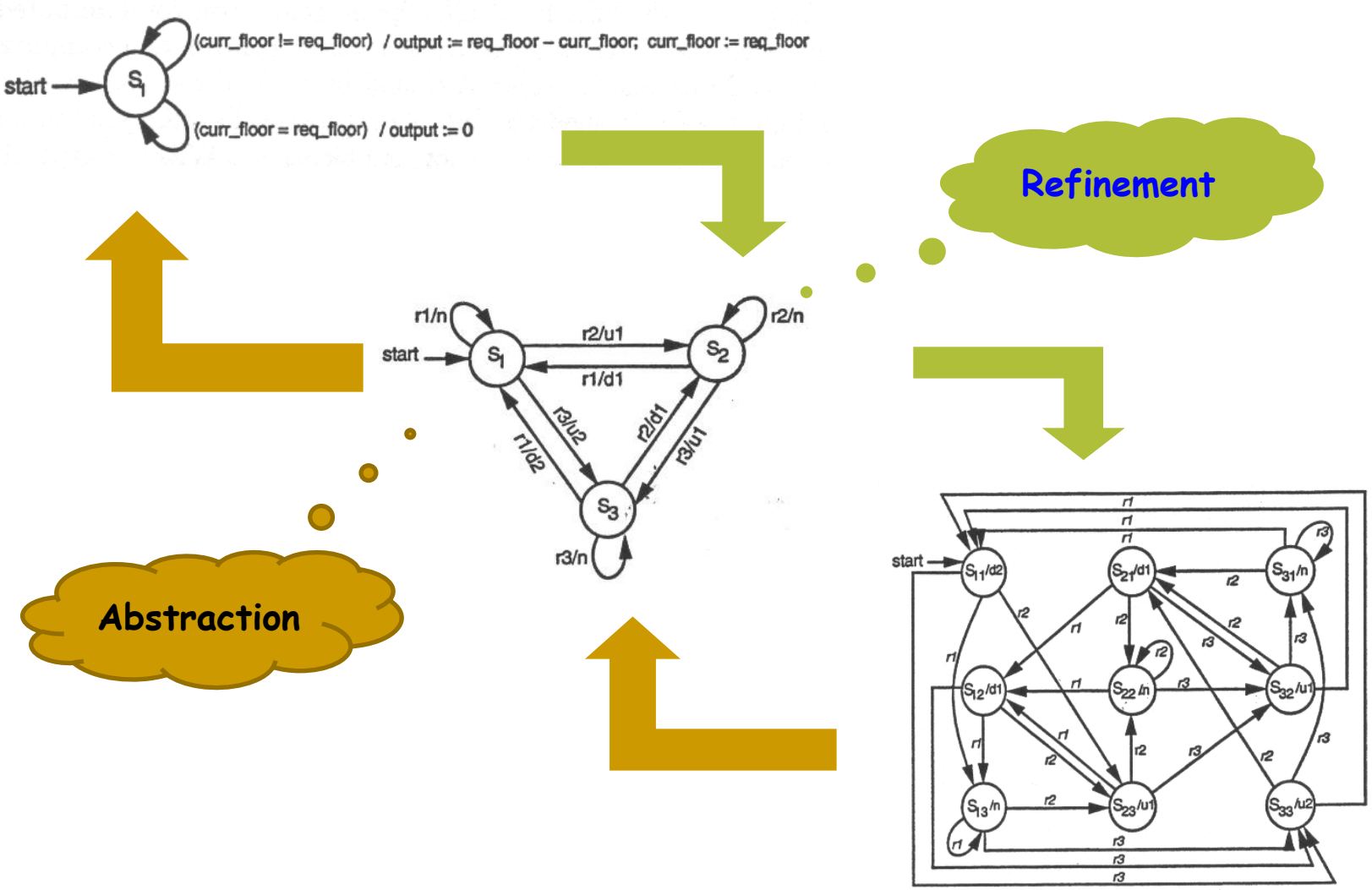
State-Orient Models

- State Machines (FSMs) M = (I, S, δ, λ, O, S_0)
  - I: set of inputs
  - S: set of states
  - O: set of outputs
  - δ: transition relation ($\delta \in S \times I \times S$)
  - λ : output relation
    - Mealy: $\lambda \in S \times I \times O$: transition-based output
    - Moore: $\lambda \in S \times O$: state-based output
  - S_0: set of initial states
  - Deterministic machines: δ and λ are functions, $S_0 = \{s_0\}$.

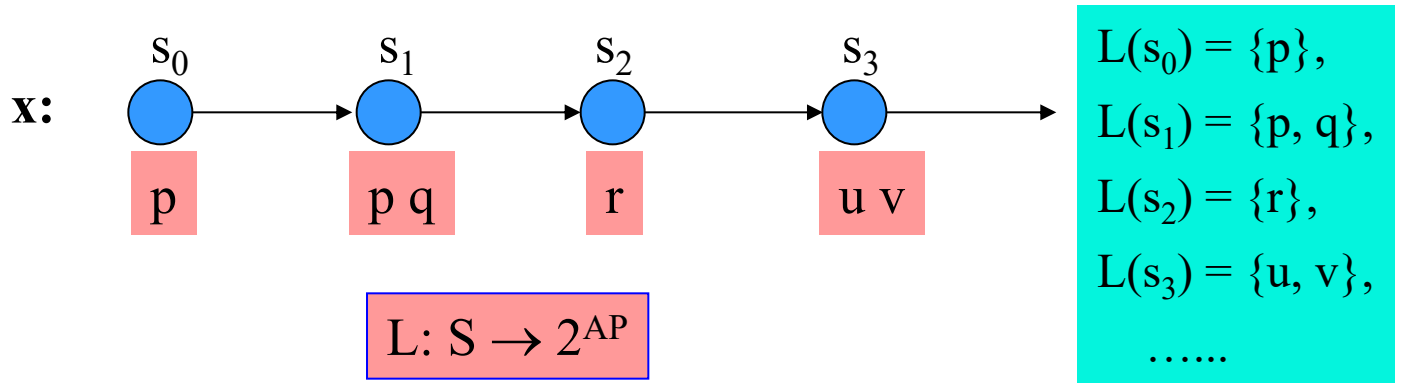


- Mostly for reactive systems
- System's state changes in response to some external events.

Abstraction & Refinement



Linear Temporal Logic



- Pnueli, 1977:
  - focus on ongoing behavior,
  - rather than input/output behavior: temporal logic

Propositional Linear Temporal Logic

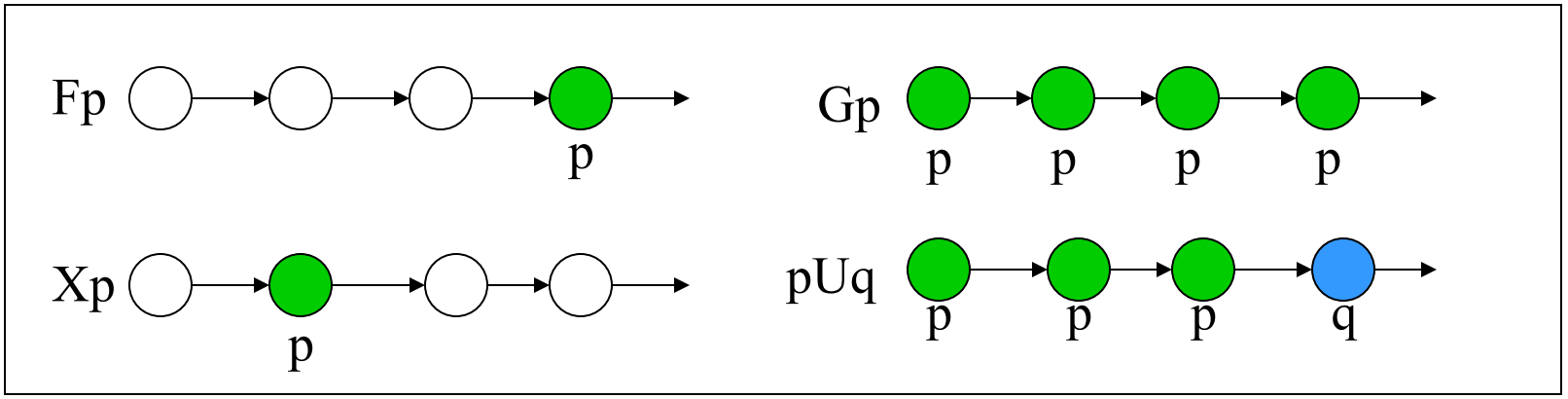
- LTL = Classical propositional logic + temporal operators
- Basic temporal operators:

  F *p* (◊p, "sometime *p*", "eventually *p*")

  G *p* (□p, "always *p*", "henceforth *p*")

  X *p* (Op, "next time *p*")

  p U *q* ("*p* until *q*")



Safety Properties

- A safety property states that some bad thing never happens.
- Safety properties represent requirements that should be continuously maintained by the system.
- They often express invariance properties.
- Examples:

  G ~(ack1 & ack2) "mutual exclusion"

  G (req →(req W ack)) "req must hold until ack"

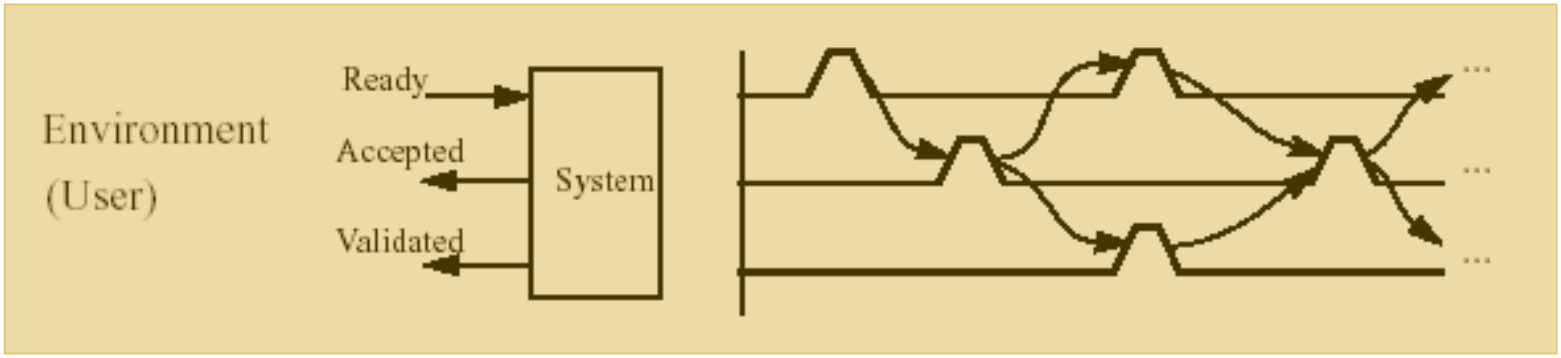
- A safety property corresponds to partial correction which does not ensure termination, but only that all terminating computations produce correct results.

Liveness Properties

- A liveness property states that some good thing eventually happens.
- Liveness properties represent requirements that need not hold continuously but those eventual (or repeated) realization must be ensured.
- Examples:

  G (req -> F ack) "if req, eventually ack"

- Liveness properties correspond to total correctness which guarantees termination.



Example: A simple interface protocol, pulses one clock period wide

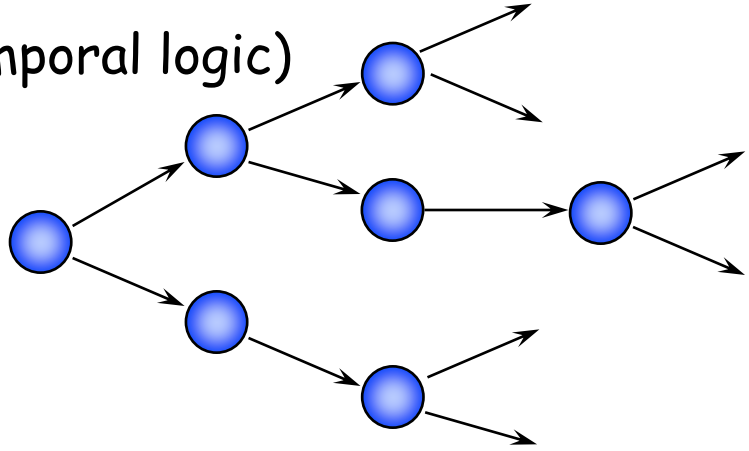
- Safety property:
  - $\forall\, t.\, (\, \mathit{Validated}\, (\, t) \rightarrow \neg\, \mathit{Validated}\, (\, t+1))$
  - $\mathbf{G}\, (\, \mathit{Validated} \rightarrow \mathbf{X}\, \neg\, \mathit{Validated})$
- Liveness:
  - $\forall\, t.\, (\, \mathit{Ready}\, (\, t\,) \rightarrow \exists (\, t' \geq t+1)\, .\, \mathit{Accepted}\, (\, t\,)\,)$
  - $\mathbf{G}\, (\, \mathit{Ready} \rightarrow \mathbf{F}\, \mathit{Accepted}\,)$
- Fairness constraint:
  - $\mathbf{G}(\, \mathit{Accepted} \Rightarrow \mathbf{F}\, \mathit{Ready}\,)$ (it models a live environment of System)

Branching Time Temporal Logic

- Structure of time
  - an infinite tree
  - each instant may have many successor instants
  - Along each path in the tree, the corresponding timeline is isomorphic to **N**

- State quantifiers:
  - Xp, Fp, Gp, pUq (like in linear temporal logic)

- Path quantifiers
  - A = "for all future paths"
  - E = "for some future path"

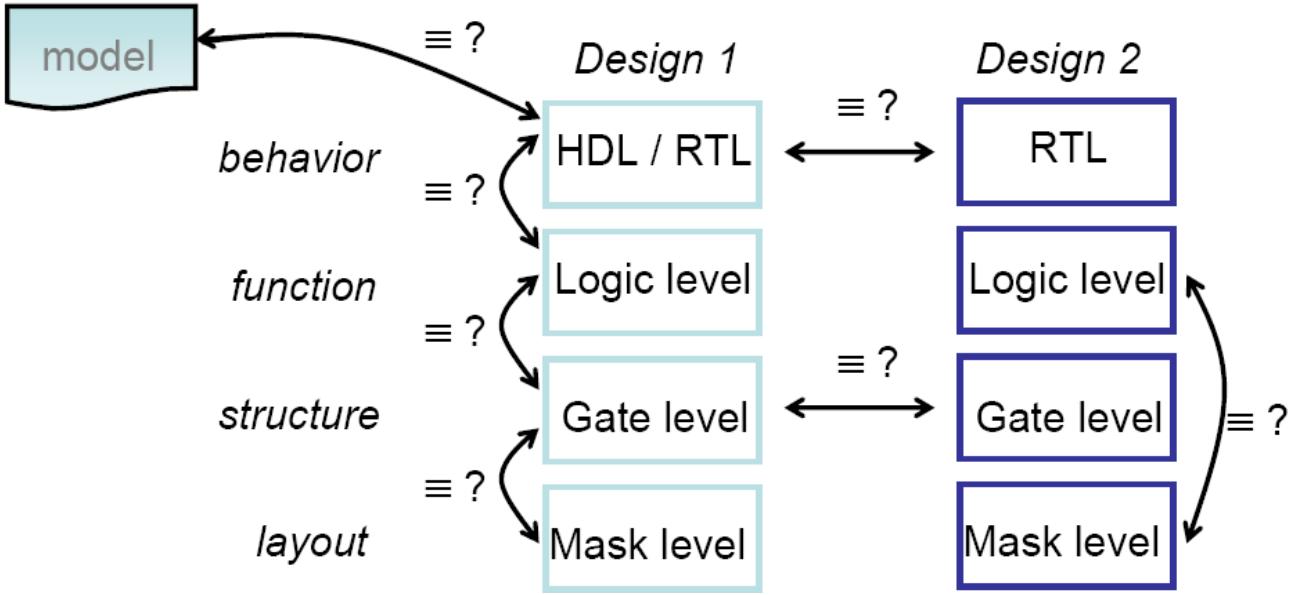


Assertion Languages

- Standardization efforts of the early 2000s by Accellera
  - PSL: temporal logic extended with regular events
  - SVA: less temporal and more regular

Automated Formal Verification

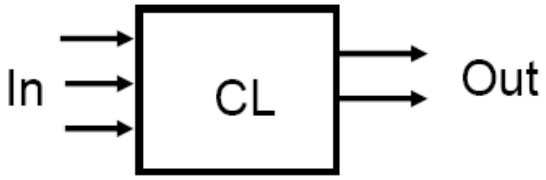
Equivalence Checking

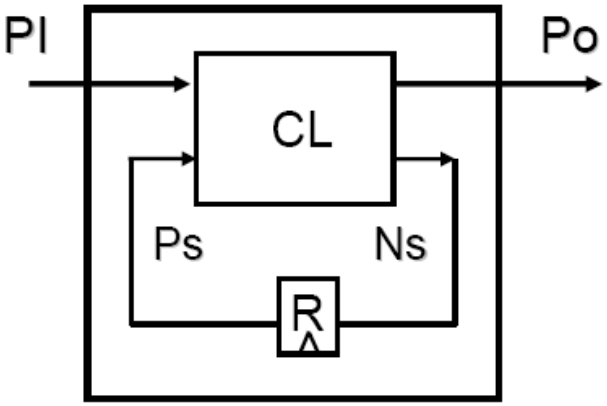


- Ensuring correctness of the design
  - Typically compare against
  - A reference model
  - an implementation (at different levels)
  - An alternative design (at the same level)

Equivalence Checking

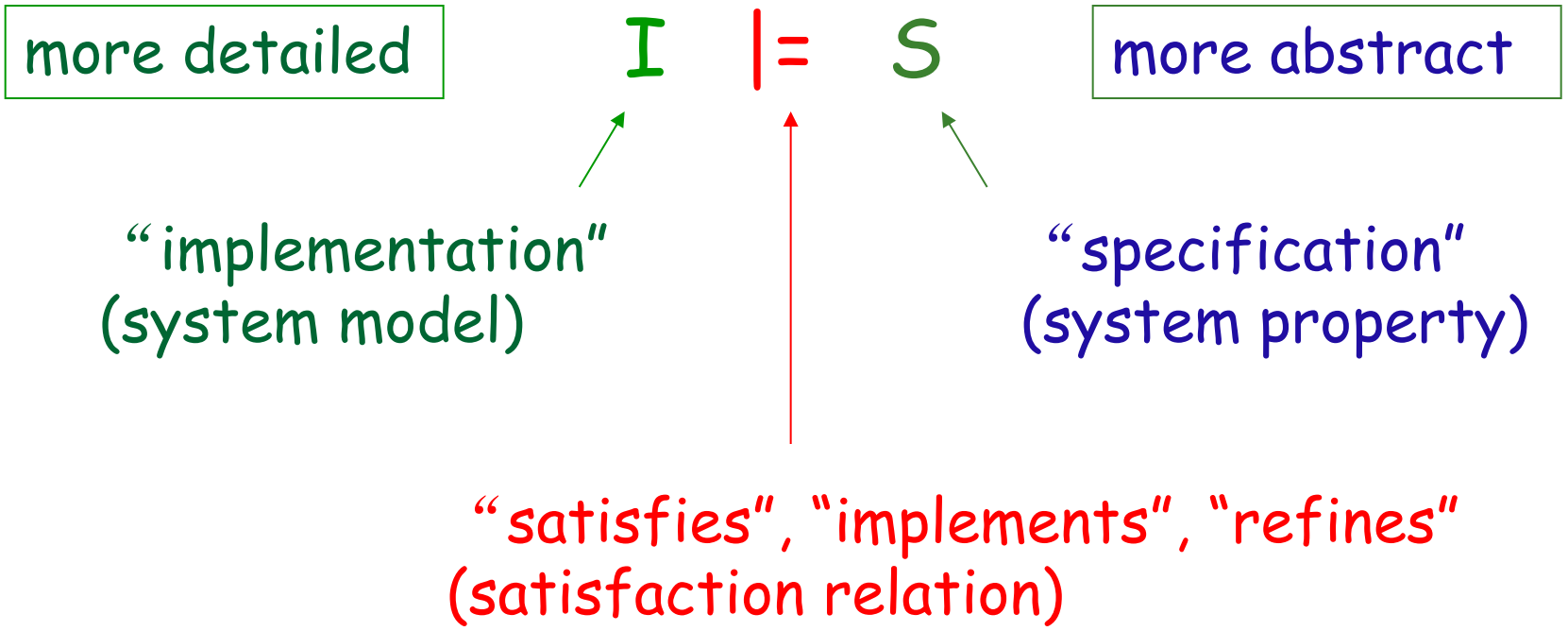
- Two circuits are functionally equivalent if they exhibit the same behavior
- Combinational circuits
  - for all possible input values
- Sequential circuits
  - for all possible
  - states & input values





Model Checking

A specific model-checking problem is defined by



Model Checking

- Model checking is a technique that relies on building a finite model of a system and checking that a desired property holds in that model
- The check is performed as an exhaustive state space search which is guaranteed to terminate since the model is finite.
- The technical challenge in model checking is in devising algorithms and data structures that allow us to handle large search spaces.

Model Checking: I

- The first temporal model checking is a technique developed independently in the 1980s by [Clarke and Emerson 1981] and by [Queille and Sifakis 1982].
  - specifications are expressed in a temporal logic [Pnueli 1981]
  - systems are modeled as finite state transition systems.
- An efficient search procedure is used to check if a given finite state transition system is a model for the specification.

Model Checking: II

- The specification is given as an automaton, then the system also modeled as an automaton is compared to the specification to determine whether or not its behavior conforms to that of the specification.
- Different notions of conformance have been explored including
  - language inclusion [Kurshan 1990]
  - refinement orderings [Cleaveland 1993]
  - observational equivalence [Cleaveland 1993]
- Vardi and Wolper [1986] showed how the temporal logic model checking problem could be recast in terms of automata, thus relating these two approaches

Model Checking

- In contrast to theorem proving model checking is completely automatic and fast sometimes producing an answer in a matter of minutes.
- Model checking can be used to check partial specifications and
  - so it can provide useful information about a systems correctness even if the system has not been completely specified.
- The *tour de force* is that it produces counterexamples which usually represent subtle errors in design and thus can be used to aid in debugging

Model Checking

- The main problem of model checking is the state explosion problem.
- In 1987, McMillan used Bryant's ordered binary decision diagrams BDDs [Bryant 1986] to represent state transition systems efficiently, thereby increasing the size of the systems that could be verified.
- Other approaches to alleviating state explosion include
  - the exploitation of partial order information [Peled 1996]
  - localization reduction [Kurshan 1994] and semantic minimization [Elseaidy 1996] to eliminate unnecessary states from a system model.

Model Checking

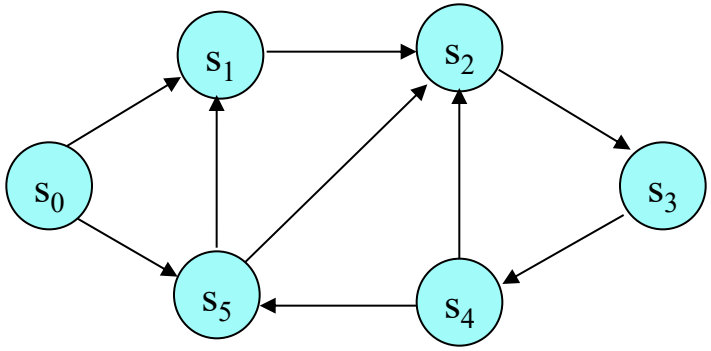
- Model checkers today are routinely expected to handle systems with between 100 and 200 state variables.
- They have checked interesting systems with 10^{120} reachable states [Burch 1994].
- By using appropriate abstraction techniques they can check systems with an essentially unlimited number of states [Clarke 1994].
- As a result, model checking is now powerful enough that it is becoming widely used in industry to aid in the verification of newly developed designs

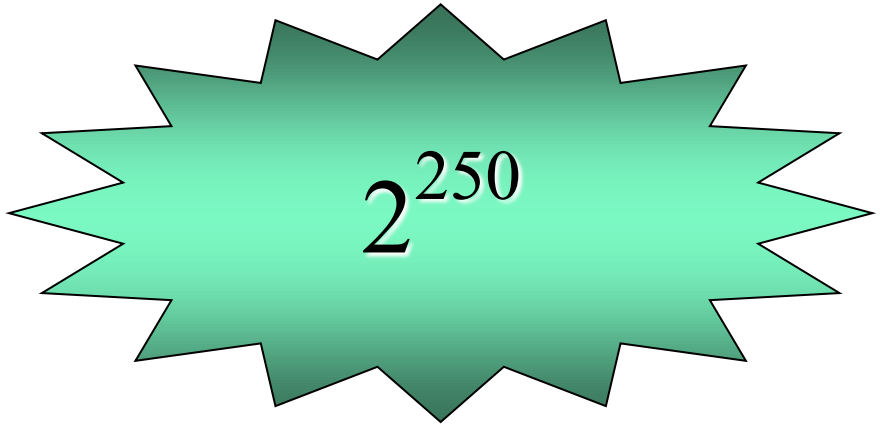
State Space Explosion

- A small design (or program) that can use a mere 250 bits of data has at least 2^{250} states

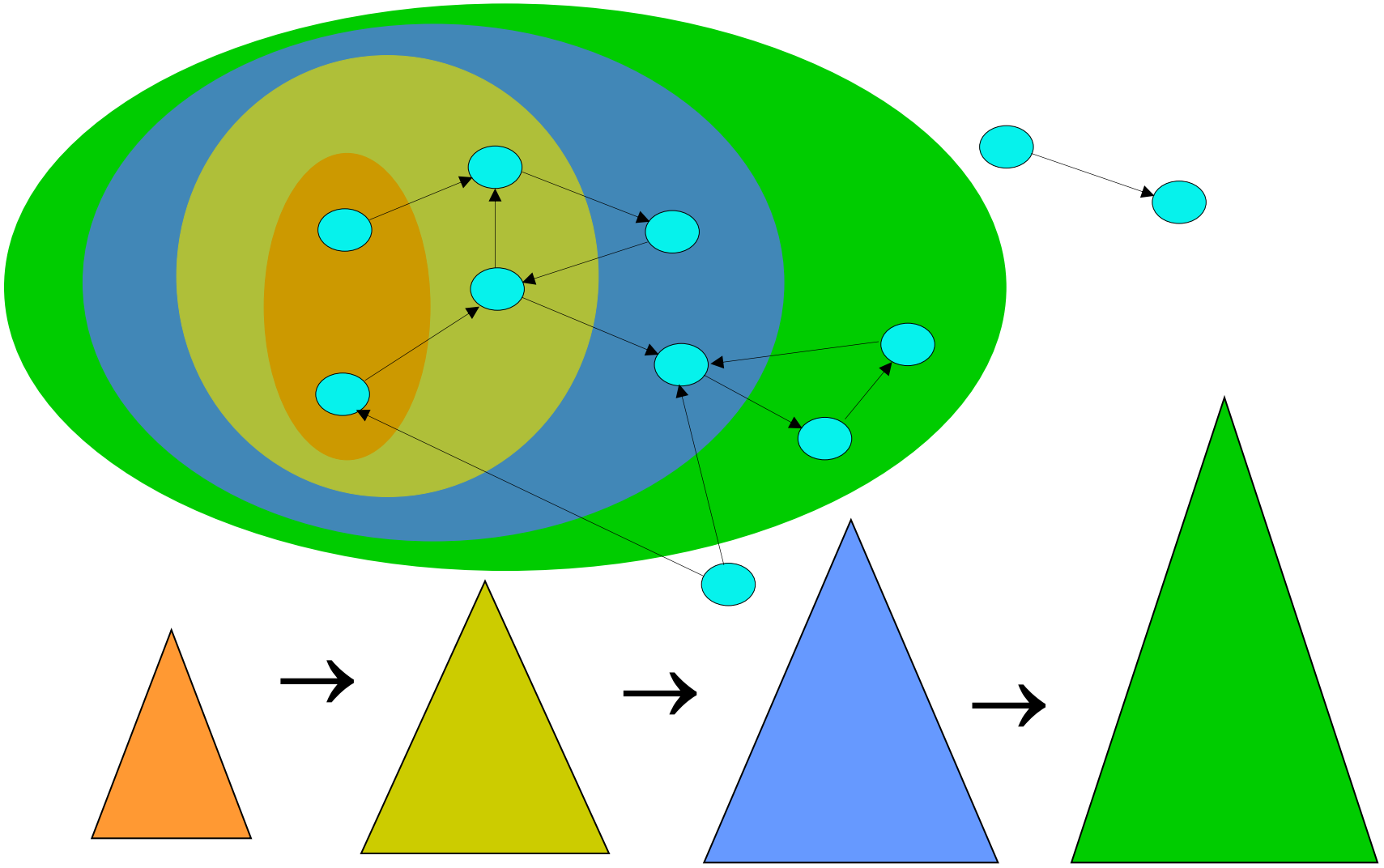
  $\Rightarrow$ More states than particles in the Universe!

- The model is too large to store in available memory!





Symbolic Image Computation

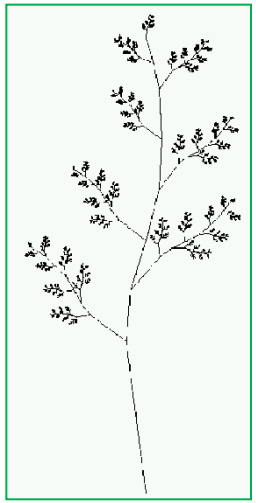


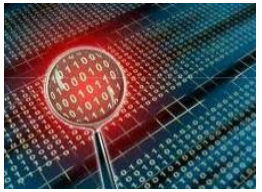
Symbolic Model Checking

CTL Formula
Kripkle Structure

Model Checker:
Fixpoint
Computation

NuSMV Package





OK /Counterexample

- Breakthrough:
- Implicit State Representation using BDDs (about 10^{20} states).

Advances in Model Checking

- Explicit model checking (1980 –)
- Symbolic Model Checking with Binary Decision Diagrams (1991 –)
- Symbolic Bounded Model Checking with SAT solvers (1999 –)
- Model Checking using SAT and Induction (2000 –)
- Unbounded SAT-based Model Checking (2003, July)

Interactive Formal Verification (Semi-automatic)

Between Spec and Imp

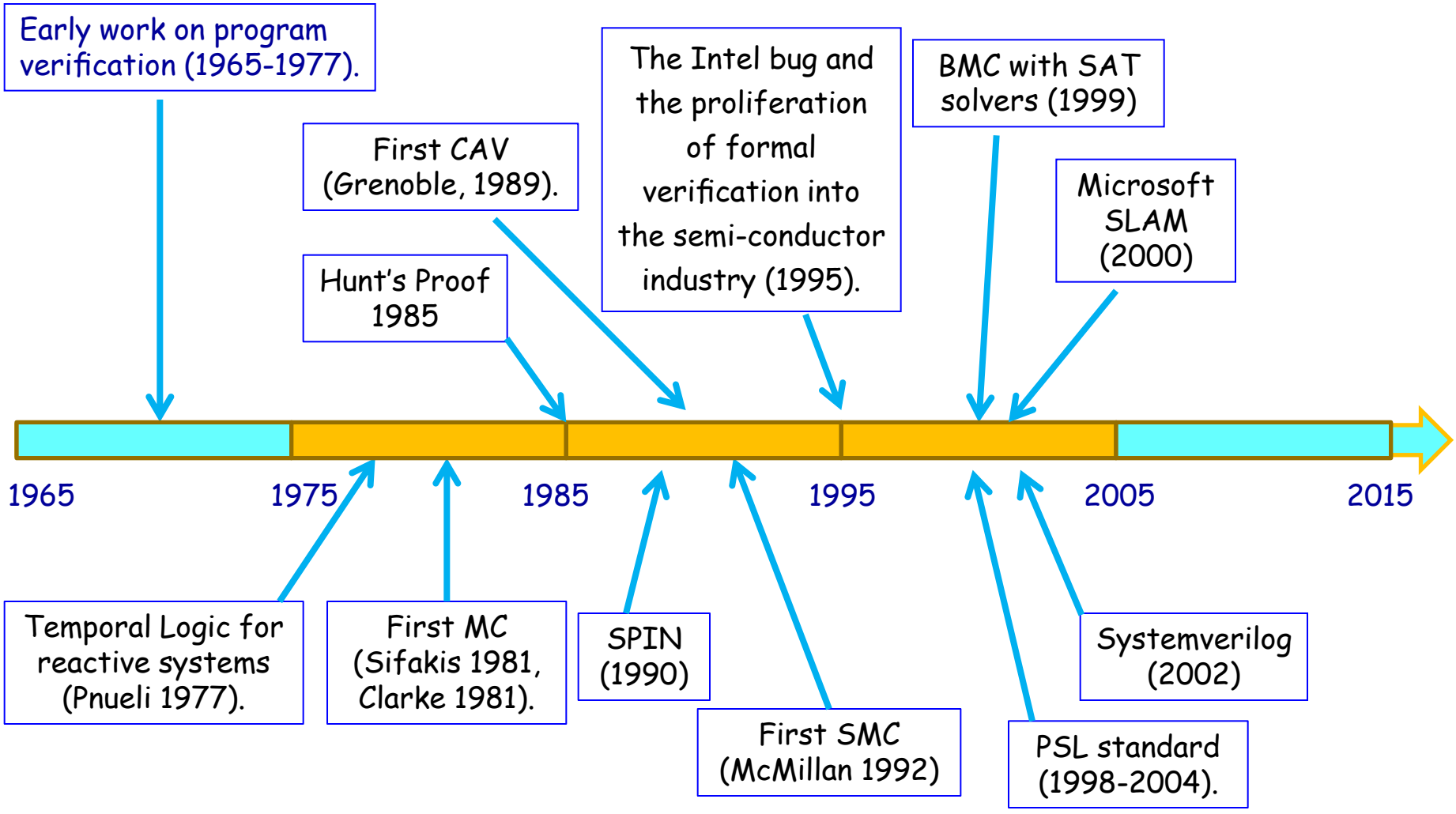
- Imp ≡ Spec
  - implementation is *equivalent* to specification
- Imp → Spec
  - implementation *logically implies* specification
- Imp |= Spec
  - implementation is a semantic model in which specification is true

Issues in Verification Methods

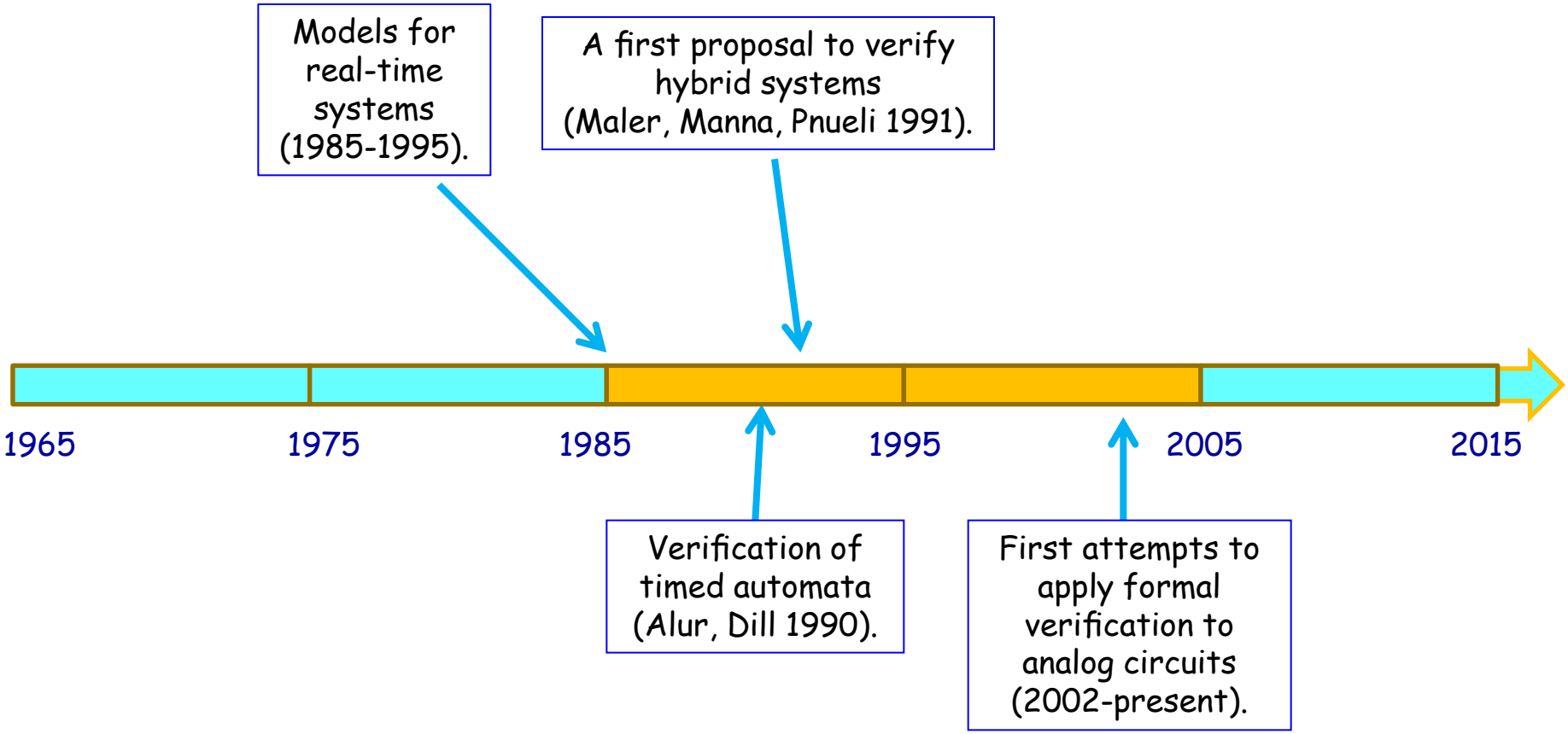
- *Automation*
  - proof generation process automatic, semi-automatic or user driven
- *Compositional*
  - constructed from proofs of component parts
- *Hierarchical*
  - system organized hierarchically at various levels of abstraction
- *Inductive*
  - reason inductively about parameterized descriptions

A History Segment

30 Years of Evolution of FV



Evolution of FV for Continuous Domains



Some Topics in This Class

Modeling for Verification

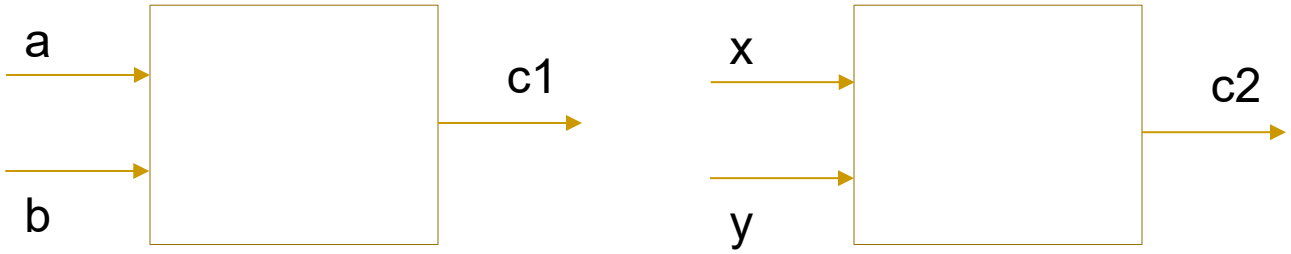
- Software
- Hardware

- Boolean Network
- FSM

Problem 1

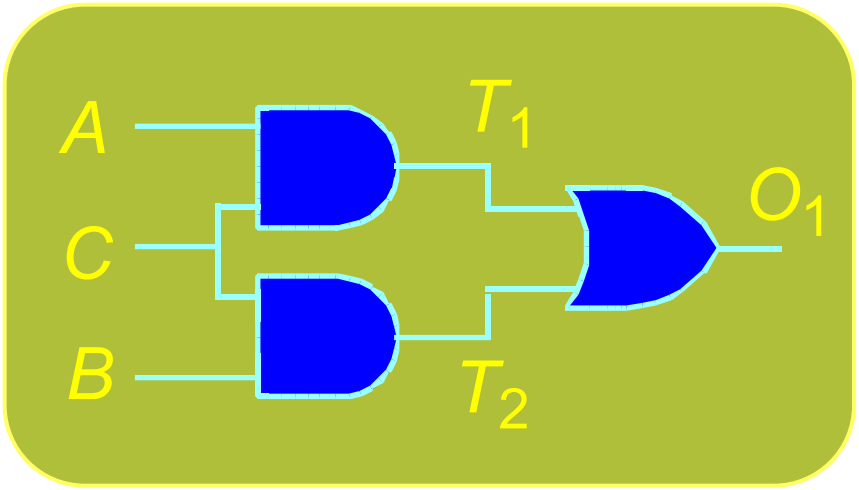
- Given two combinational circuits C1 of N1 gates and C2 of N2 gates, how can we know that they are equivalent?
  - By simulation?
  - By formal verification?
- Automatically?
  - N1 and N2 are small.
  - N1 and N2 are large.

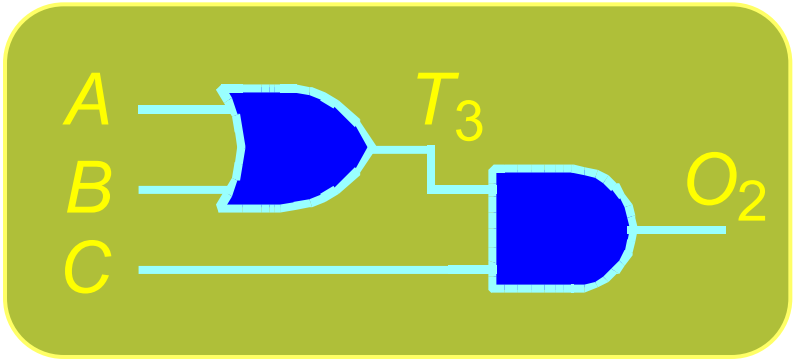
Combinational Equivalence Checking



- Suppose we have two combinational circuits C1 and C2.
- If you use simulation, how can you justify C1=C2?
- List all the issues you may think of.
- Have an example to show the idea
- Is it complete?

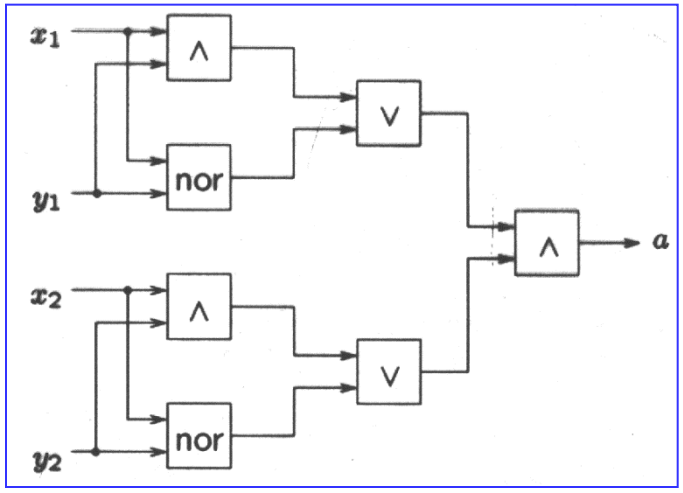
Equivalent?

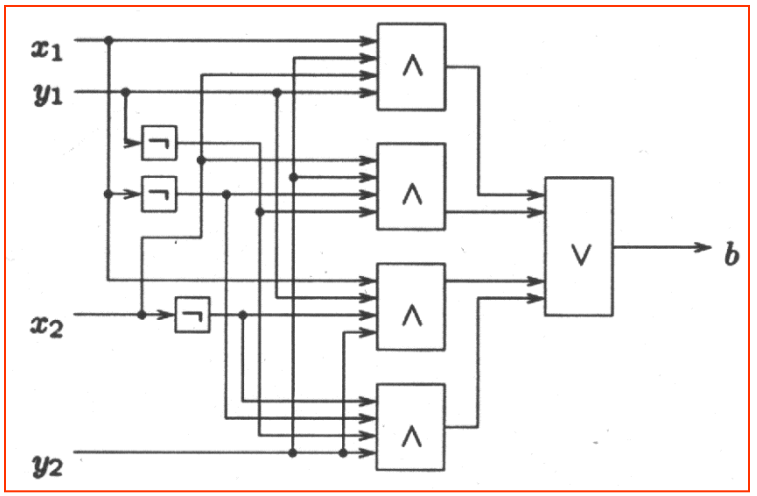




- Logic Circuit Comparison
  - Do circuits compute identical function?
  - Basic task of formal hardware verification
  - Compare new design to "known good" design

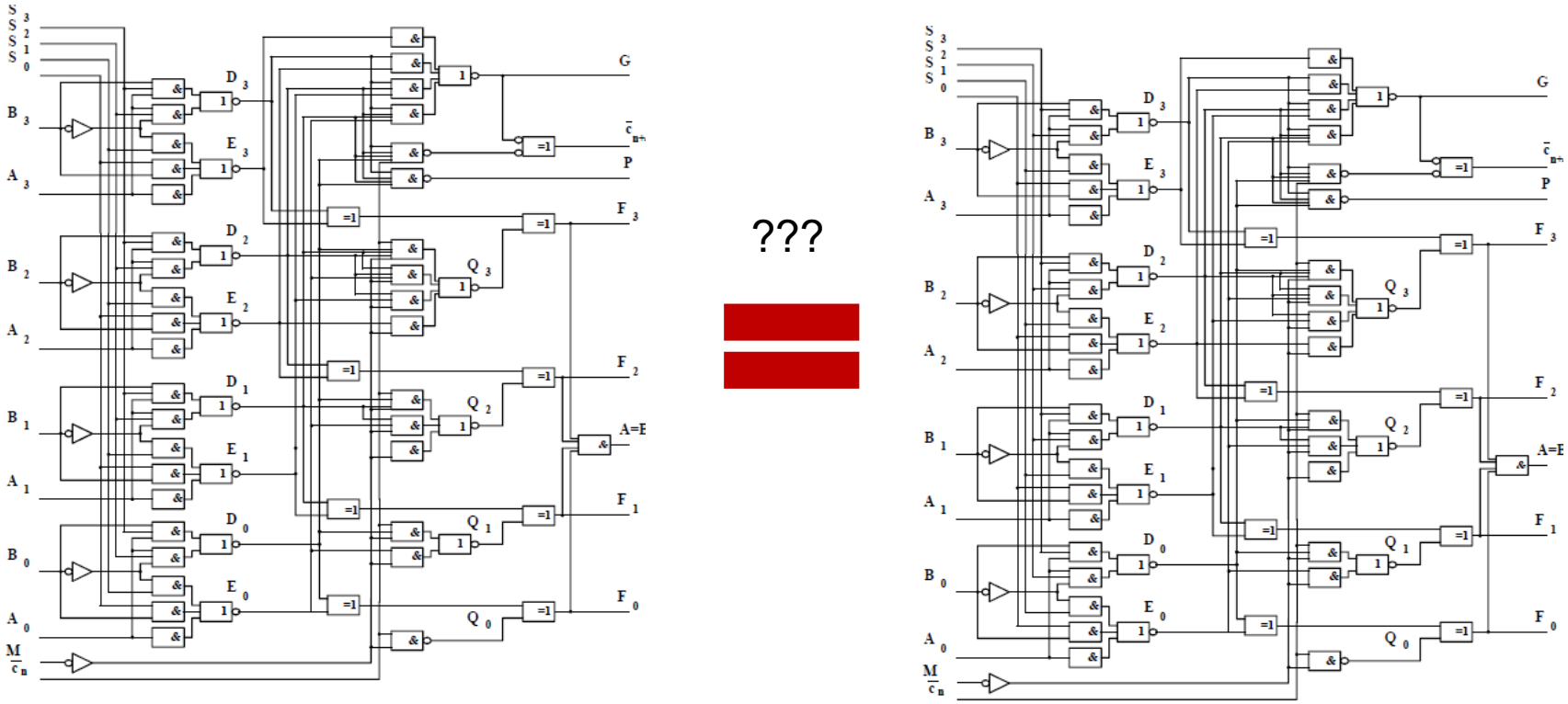
Combinational Equivalence Checking



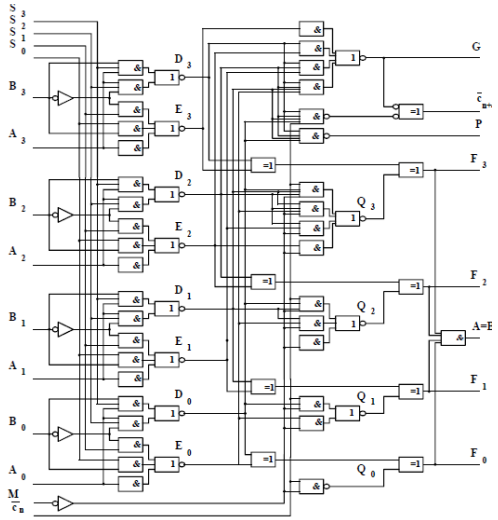


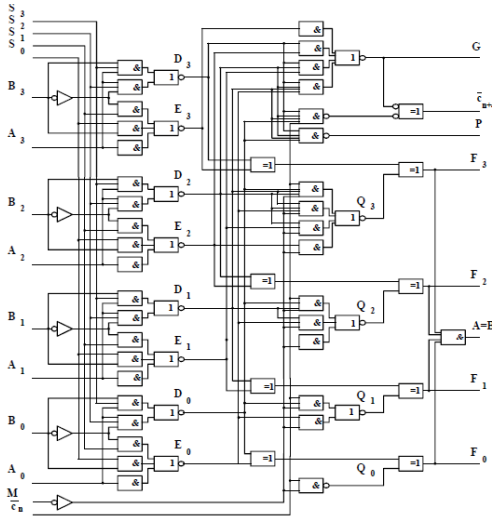
- Given: Combinational circuits C1 and C2
- (Boolean functions B1 and B2)
- How can we prove that C1 is/isn't equivalent to C2,
  - in a reasonable amount of time?

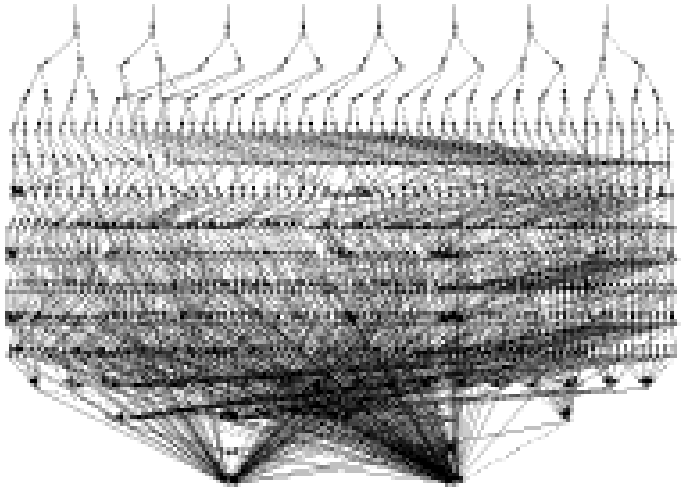
Equivalence Checking (SN 74181 ALU)

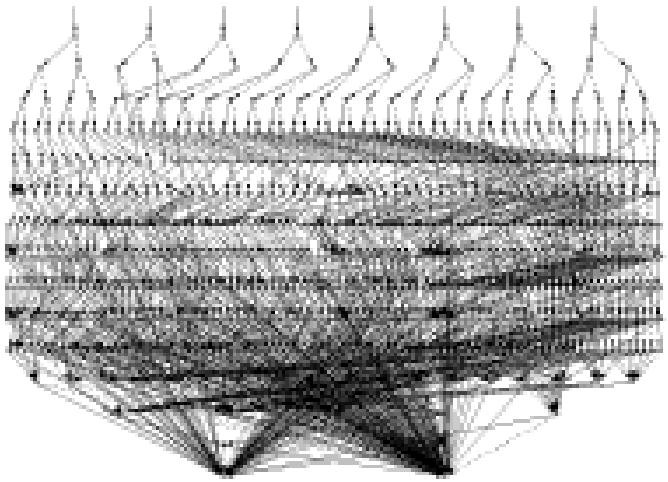


Equivalence Checking





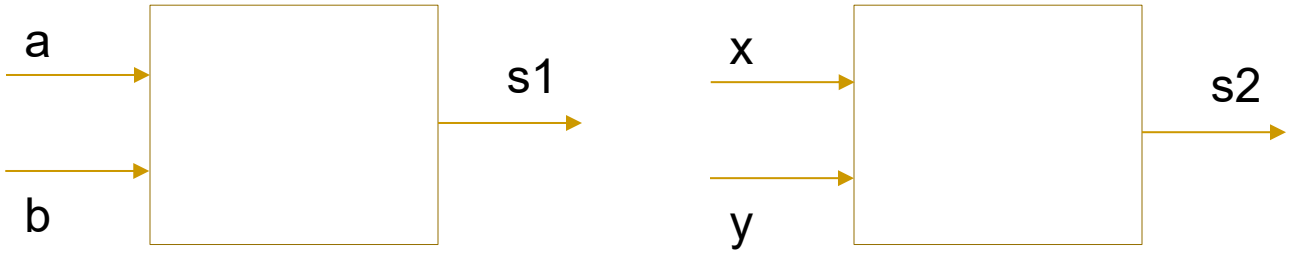




Problem 2

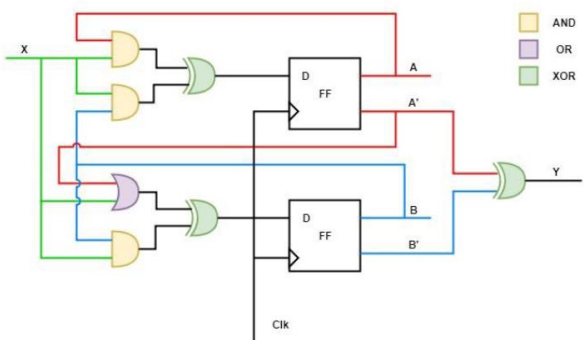
- Given two sequential circuits S1 of N1 FFs and S2 of N2 FFs, how can we know that they are equivalent?
  - By simulation?
  - By formal verification?
- Automatically?
  - N1 and N2 are small.
  - N1 and N2 are large.

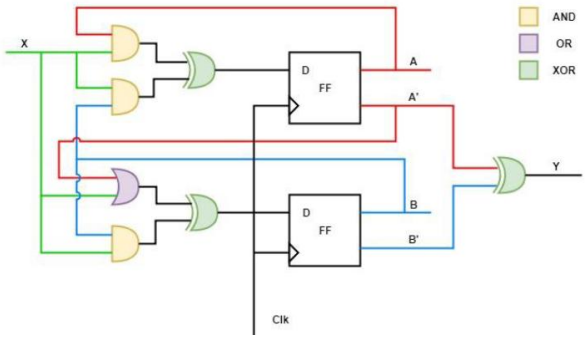
Sequential Equivalence Checking

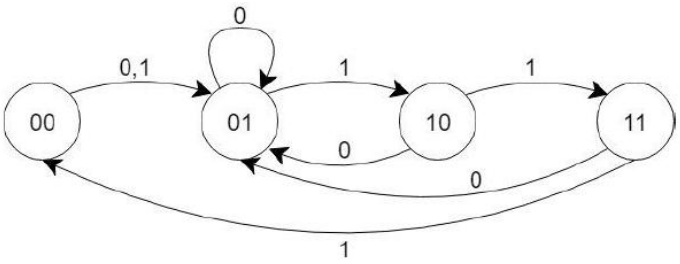


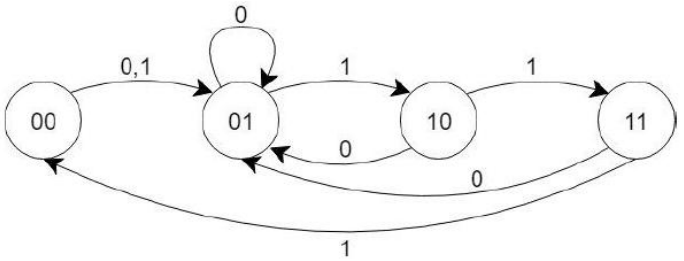
- Suppose we have two sequential circuits S1 and S2.
- If you use simulation, how can you justify S1=S2?
- List all the issues you may think of.
- Have an example to show the idea.

Sequential Equivalence Checking

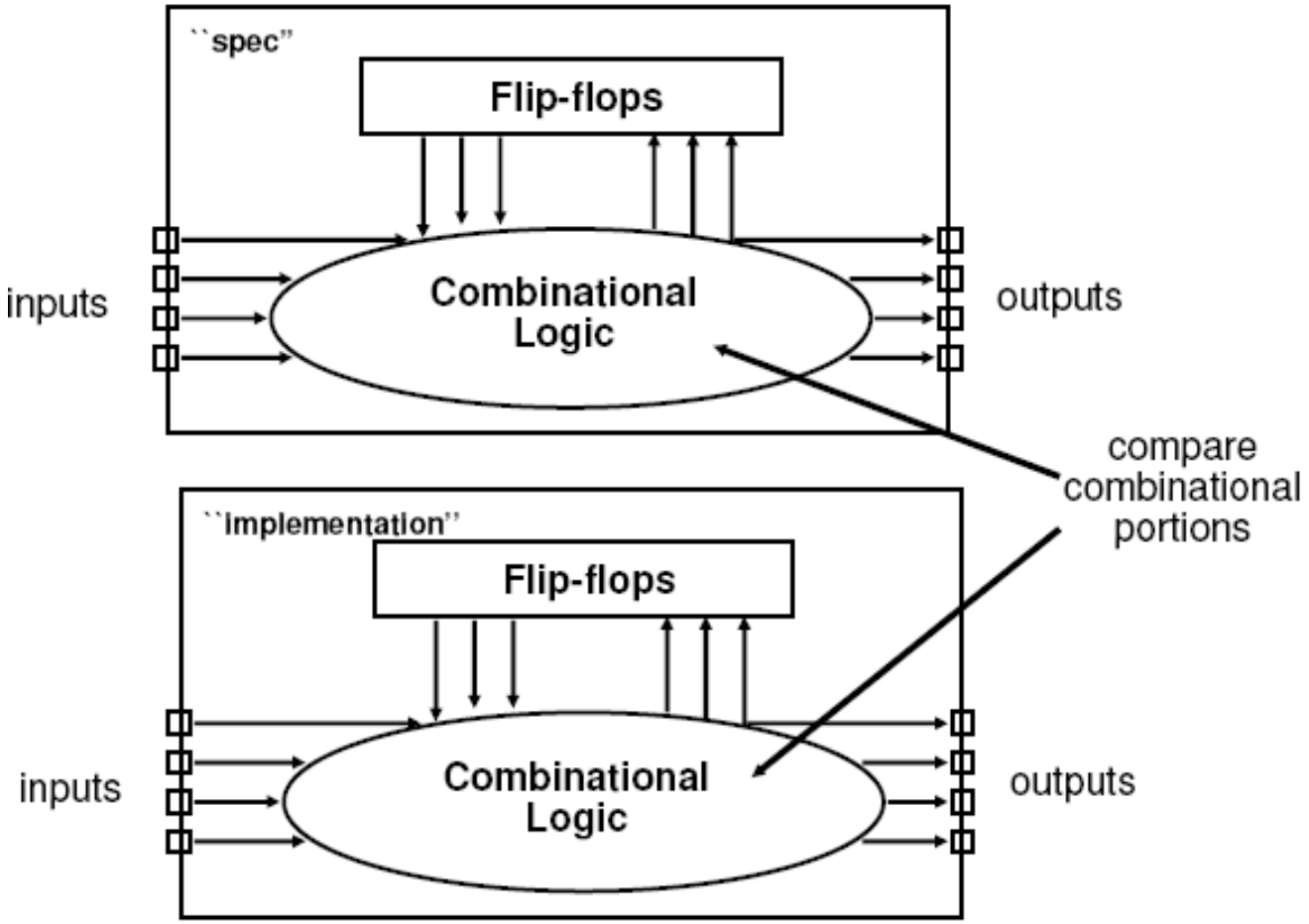








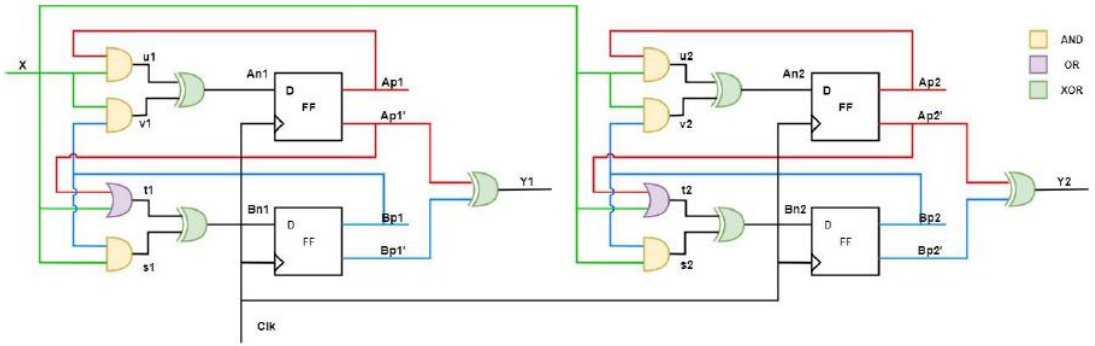
Extract Combinational Portions

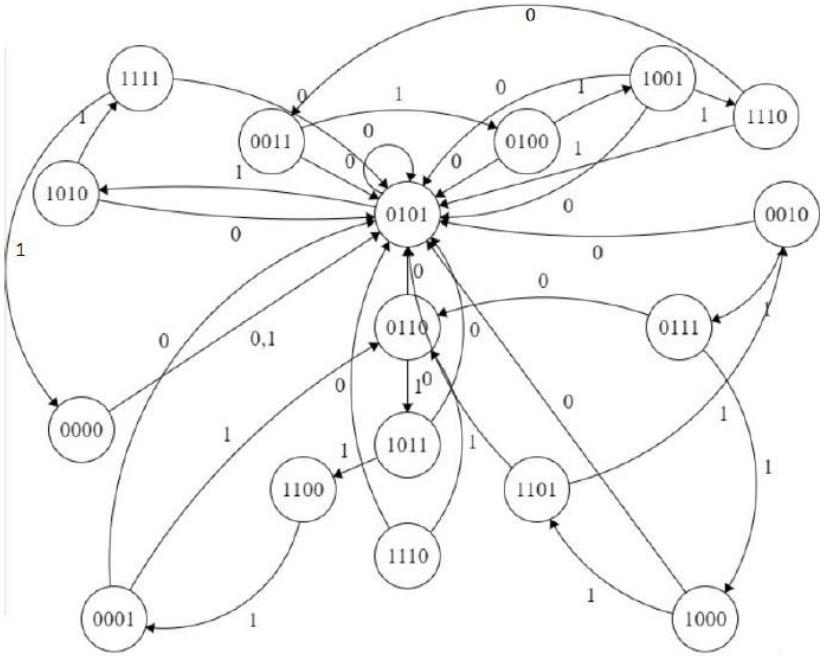


Problem 3

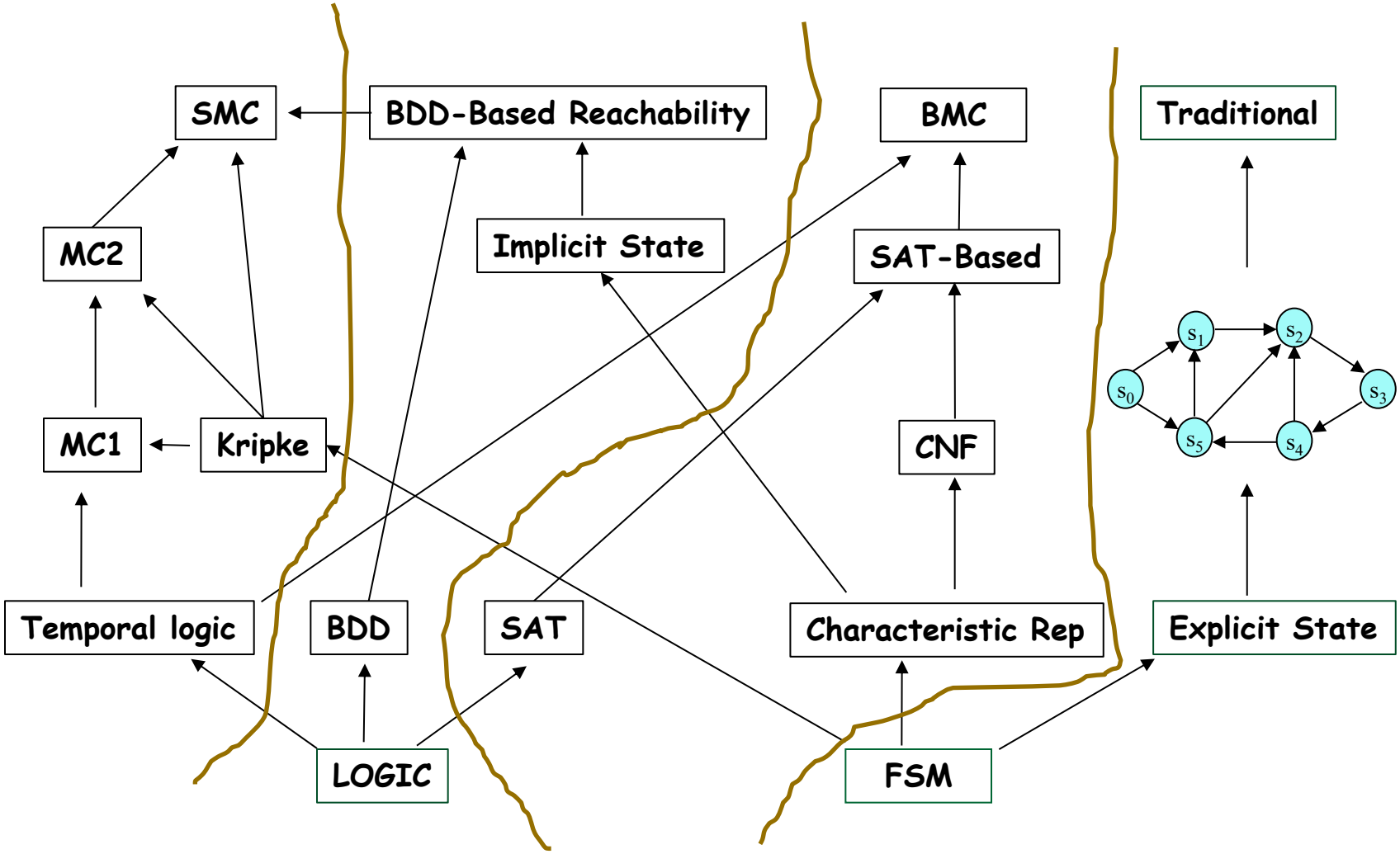
- Given FSM M of N states, how can we know that the output has some value at a state?
  - By simulation?
  - By formal verification?
- Automatically?
  - N is small.
  - N is large.

Model Checking

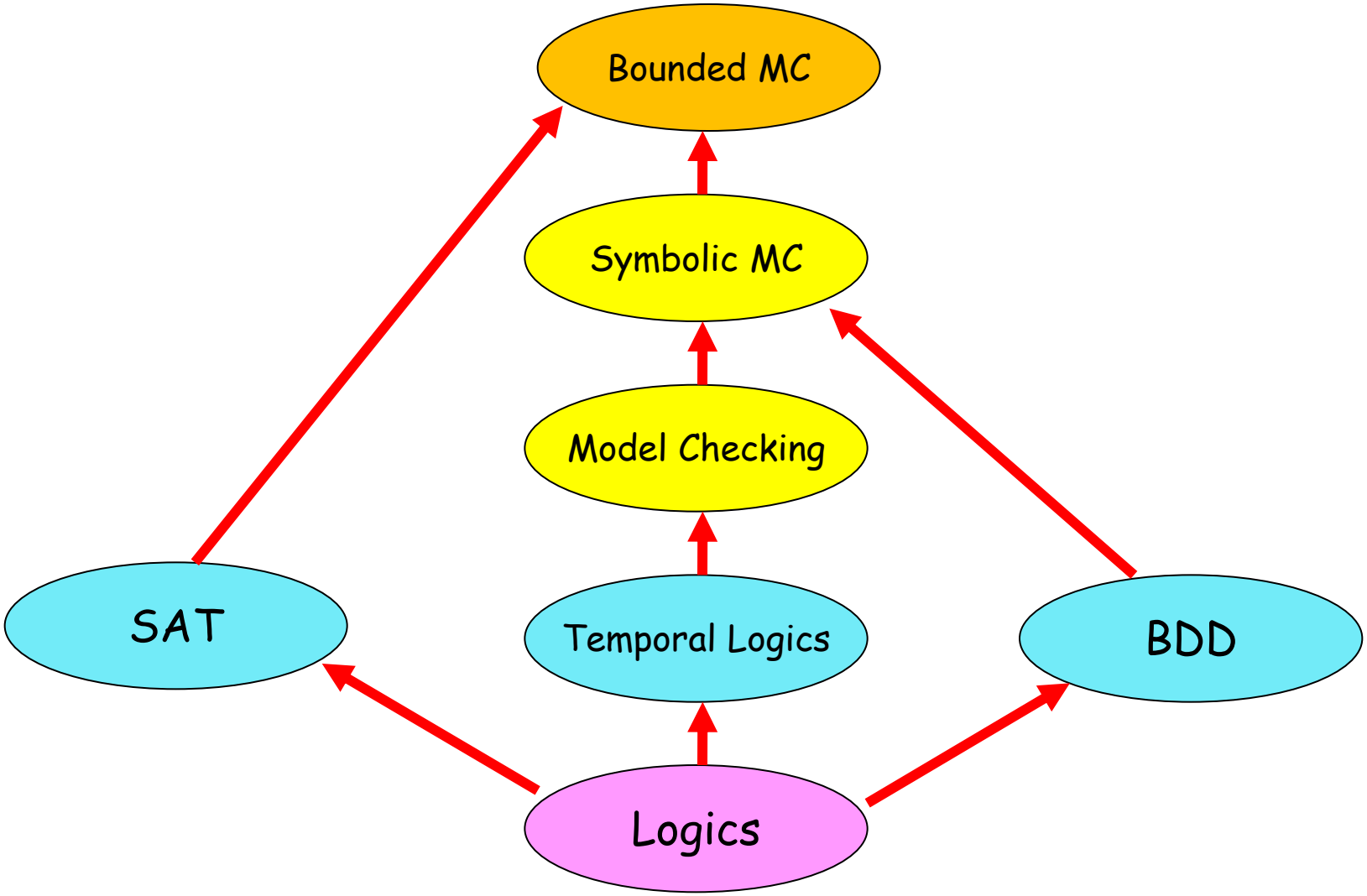


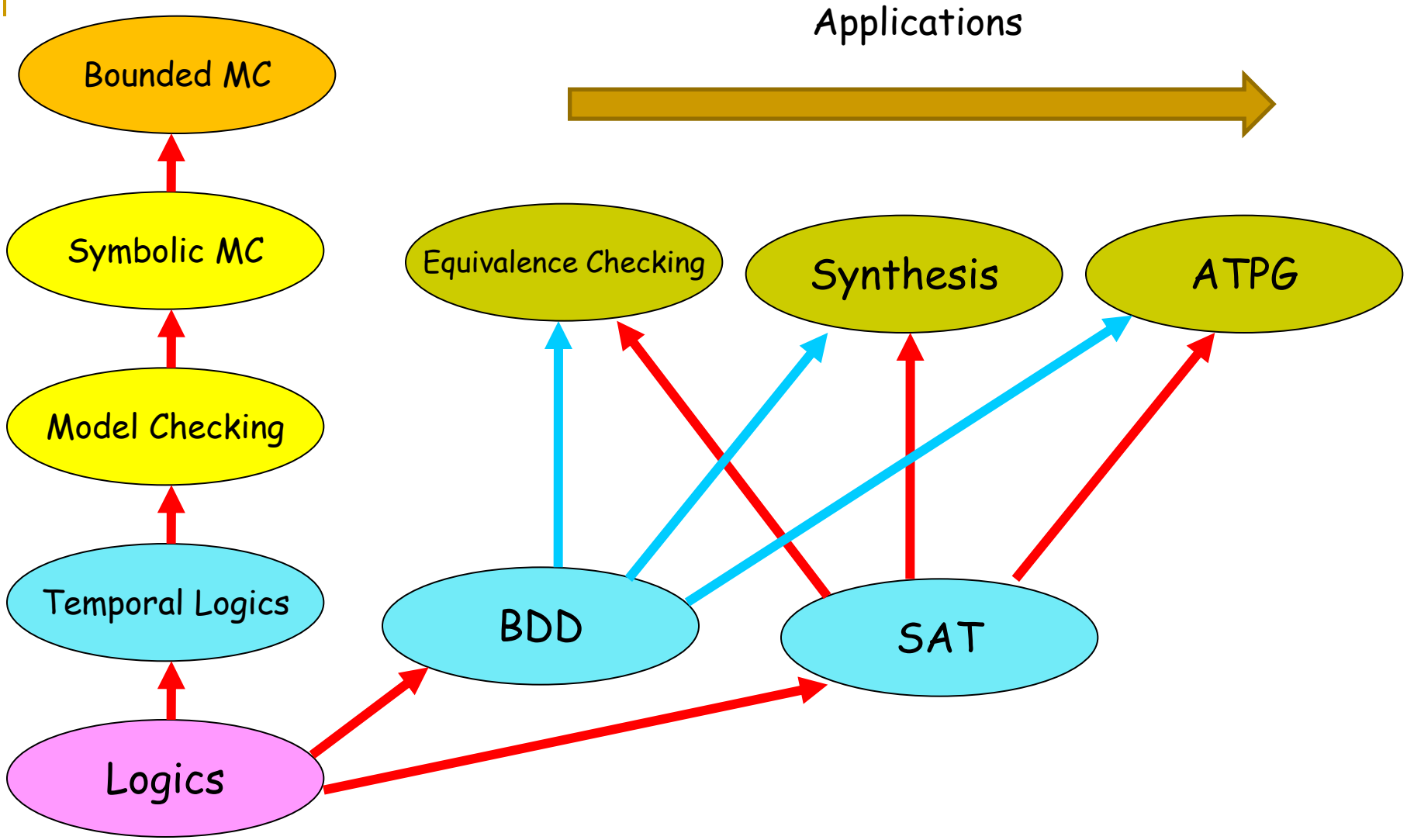


A Road Map


SMC
BDD-Based Reachability
BMC
Traditional
MC2
Implicit State
SAT-Based
MC1
Kripke
CNF
Temporal logic
BDD
SAT
Characteristic Rep
Explicit State
LOGIC
FSM
s0
s1
s2
s3
s4
s5

Where to Go?


Bounded MC
Symbolic MC
Model Checking
SAT
Temporal Logics
BDD
Logics


Applications
Bounded MC
Symbolic MC
Model Checking
Temporal Logics
Logics
Equivalence Checking
Synthesis
ATPG
BDD
SAT

Reading Materials

1. C. Kern and M. Greenstreet, "Formal Verification in Hardware Design: A Survey", ACM Transactions on Design Automation of E. Systems, Vol. 4, April 1999, pp. 123-193.
2. Aarti Gupta, "Formal Hardware Verification Methods: A Survey", Formal Methods in System Design, Vol. 1, pp. 151-238, 1992.